UC | Facultad de Ciencias

Análisis de sentimiento de textos cortos enriquecidos con información visual de seguimiento ocular

Sentiment analysis of short texts enhanced with eye-tracking visual information

Trabajo de Fin de Máster para acceder al

MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA

Autor: JUNCAL BLANCO, AITOR

Director: PALAZUELOS CALDERÓN, CAMILO

Codirector: DUQUE MEDINA, RAFAEL

Cantabria, septiembre 2025

Índice general

1. Introducción **3**
- 1.1. Motivación . . . 3
- 1.2. Objetivos . . . 4
- 1.3. Estructura del resto del documento . . . 5

2. Materiales y métodos **6**
- 2.1. Eye-tracker . . . 6
- 2.2. Análisis de sentimiento . . . 8
- 2.3. Trabajos relacionados . . . 8

3. Generación del conjunto de datos **10**
- 3.1. Captura de datos . . . 10
- 3.2. Extracción de fotogramas clave . . . 10
- 3.3. Normalización y recorte de imágenes . . . 11
- 3.4. Generación del conjunto de datos textual a partir de información visual 12
  - 3.4.1. Extracción del texto . . . 12
  - 3.4.2. Cálculo del score de atención . . . 13
  - 3.4.3. Contenido del fichero CSV resultante . . . 15
  - 3.4.4. Dos análisis posibles . . . 15

4. Metodología **16**
- 4.1. Preprocesamiento . . . 16
- 4.2. Construcción de los grafos de polaridad . . . 17
- 4.3. Ajuste del modelo de red neuronal . . . 20

5. Experimentación **22**
- 5.1. Estructura del conjunto de datos . . . 22
- 5.2. Distribución de los datos . . . 23
  - 5.2.1. Conjunto de datos del análisis basado en respuestas . . . 24
  - 5.2.2. Conjunto de datos del análisis de sentimiento puro . . . 25
- 5.3. Cálculo de métricas de rendimiento . . . 25
- 5.4. Selección de parámetros y estudio de ablación . . . 25
- 5.5. Comparación con modelos de lenguaje BERT . . . 26

6. Resultados **27**
- 6.1. Presentación de los resultados con nuestro modelo . . . 27
- 6.2. Selección de parámetros y estudio de ablación . . . 28
  - 6.2.1. Análisis basado en respuestas . . . 28
  - 6.2.2. Análisis de sentimiento puro . . . 32

6.2.3. Discusión 36
6.3. Comparación con otros trabajos 36
6.3.1. Análisis basado en respuestas 37
6.3.2. Análisis de sentimiento puro 39

7. Conclusiones y trabajo futuro **42**
7.1. Conclusiones 42
7.2. Líneas de trabajo futuro 43

Anexo **45**

Índice de tablas

2.1. Especificaciones del sistema de seguimiento ocular (Pro, 2019) 7
2.2. Resumen de características extraídas de estudios con *eye-tracking* para clasificación emocional, según Lim et al. (2020). 9

5.1. Distribución de ejemplos positivos y negativos en entrenamiento y test para cada fold. 25
5.2. Distribución de ejemplos positivos y negativos en entrenamiento y test para cada fold. 25

6.1. Ejemplo de métricas resultantes para diferentes combinaciones de Alpha y Beta 28
6.2. Modelos BERT elegidos 36

Índice de figuras

2.1. Eyetracker Tobii Pro Nano (Pro, 2019) 7

3.1. Ejemplo de un fotograma con un tweet y su respuesta 11
3.2. Ejemplo del mapa de calor asociado 12
3.3. Ejemplo de un fotograma al que se le ha aplicado OCR 13
3.4. Coordenadas OCR superpuestas en el mapa de calor 13

4.1. Frase *Mañana jueves lloverá mucho* con distancia = 1 19
4.2. Frase *Mañana jueves lloverá mucho* con distancia = 2 19
4.3. Frase *Mañana jueves lloverá mucho* con distancia = 3 19

5.1. Resultado de dividir el conjunto de datos inicial en 15 bloques 23
5.2. Resultado de dividir los 15 bloques en 3 folds 23
5.3. Cada fold tiene a su vez un conjunto de entrenamiento y de test . . . 23
5.4. Número de palabras para los 15 tweets originales 24
5.5. Distribución de palabras para las respuestas 24

6.1. Gráfica que muestra la evolución para el *AUC* 29
6.2. Gráfica que muestra la evolución para el *Accuracy* 29
6.3. Gráfica que muestra la evolución para la precisión negativa 30
6.4. Gráfica que muestra la evolución para la precisión positiva 30
6.5. Gráfica que muestra la evolución para el *recall* negativo 31
6.6. Gráfica que muestra la evolución para el *recall* positivo 31
6.7. Gráfica que muestra la evolución para el *F1-Score* negativo 31
6.8. Gráfica que muestra la evolución para el *F1-Score* positivo 32
6.9. Gráfica que muestra la evolución para el *AUC* 33
6.10. Gráfica que muestra la evolución para el *Accuracy* 33
6.11. Gráfica que muestra la evolución para la precisión negativa 34
6.12. Gráfica que muestra la evolución para la precisión positiva 34
6.13. Gráfica que muestra la evolución para el *recall* negativo 34
6.14. Gráfica que muestra la evolución para el *recall* positivo 35
6.15. Gráfica que muestra la evolución para el *F1-Score* negativo 35
6.16. Gráfica que muestra la evolución para el *F1-Score* positivo 35
6.17. Gráfica que muestra la evolución para el *AUC* 37
6.18. Gráfica que muestra la evolución para la *Accuracy* 37
6.19. Gráfica que muestra la evolución para la precisión negativa 38
6.20. Gráfica que muestra la evolución para la precisión positiva 38
6.21. Gráfica que muestra la evolución para el *recall* negativo 38
6.22. Gráfica que muestra la evolución para el *recall* positivo 38

6.23. Gráfica que muestra la evolución para el *F1-Score* negativo 39
6.24. Gráfica que muestra la evolución para el *F1-Score* positivo 39
6.25. Gráfica que muestra la evolución para el *AUC* 39
6.26. Gráfica que muestra la evolución para la *Accuracy* 39
6.27. Gráfica que muestra la evolución para la precisión negativa 40
6.28. Gráfica que muestra la evolución para la precisión positiva 40
6.29. Gráfica que muestra la evolución para el *recall* negativo 40
6.30. Gráfica que muestra la evolución para el *recall* positivo 40
6.31. Gráfica que muestra la evolución para el *F1-Score* negativo 41
6.32. Gráfica que muestra la evolución para el *F1-Score* positivo 41

Resumen

El análisis de sentimiento (también conocido como minería de opinión) consiste en detectar información subjetiva en textos e imágenes mediante modelos de aprendizaje automático y procesamiento del lenguaje natural. Una de las tareas clásicas del análisis de sentimiento es identificar la polaridad de un texto, clasificándolo en al menos dos categorías: positiva o negativa. En los últimos años, con el auge de los grandes modelos de lenguaje preentrenados, las propuestas de análisis de sentimiento han tendido hacia enfoques de caja negra con escasa o nula interpretabilidad. Como respuesta, han surgido modelos más transparentes que priorizan la representación semántica del texto para ponderar la importancia de cada palabra o frase en la clasificación. Sin embargo, estos modelos suelen limitarse a procesar únicamente texto y, en menor medida, información visual como imágenes.

En este trabajo, proponemos un modelo basado en dos principios fundamentales. Primero, que sea sencillo e interpretable, lo que permitirá una mayor rapidez en las ejecuciones y facilitará la comprensión de su funcionamiento. Segundo, que sea más completo en cuanto a las fuentes de información, combinando texto con datos derivados de imágenes en forma de mapas de calor, los cuales reflejan la interacción visual del usuario con el contenido a analizar. Dicha información será medida mediante técnicas de rastreo ocular, con la que se puede definir que áreas han suscitado más o menos atención al usuario.

Para llevar a cabo el trabajo, recopilamos una serie de tweets junto con vídeos de usuarios que respondían a ellos en tiempo real. De cada vídeo se extrajeron fotogramas que muestran el tweet y la respuesta escrita, asociando a cada uno un mapa de calor que refleja la atención visual del usuario. Posteriormente, mediante técnicas de extracción de texto en imágenes, construimos un conjunto de datos textual enriquecido con información visual, dividido en entrenamiento y test. Sobre el conjunto de entrenamiento generamos dos grafos de polaridad (positivo y negativo) y entrenamos un modelo capaz de calcular métricas de similitud entre nuevos textos y dichos grafos. Finalmente, aplicamos el modelo al conjunto de test para predecir la polaridad de textos desconocidos y evaluar así su precisión y capacidad de generalización.

PALABRAS CLAVE: rastreo ocular, mapas de calor, sentimiento, modelo, predicción.

Abstract

Sentiment analysis (also known as opinion mining) consists of detecting subjective information in texts and images through machine learning models and natural language processing. One of the classical tasks of sentiment analysis is identifying the polarity of a text, classifying it into at least two categories: positive or negative. In recent years, with the rise of large pretrained language models, sentiment analysis approaches have tended to rely on black-box methods with little or no interpretability. In response, more transparent models have emerged, prioritizing the semantic representation of text in order to weigh the importance of each word or phrase in classification. However, these models usually limit themselves to processing only text and, to a lesser extent, visual information such as images.

In this work, we propose a model based on two fundamental principles. First, it should be simple and interpretable, which allows for faster executions and facilitates understanding of its functioning. Second, it should be more comprehensive regarding sources of information, combining text with data derived from images in the form of heatmaps, which reflect the user's visual interaction with the content under analysis. Such information will be measured through eye-tracking techniques, which make it possible to define which areas attracted more or less attention from the user.

To achieve this work, we collected a series of tweets along with videos of users responding to them in real time. From each video, we extracted frames showing the tweet and the written response, associating each one with a heatmap that reflects the user's visual attention. Using image-based text extraction techniques, we then built a textual dataset enriched with visual information, which was divided into training and test sets. On the training set, we generated two polarity graphs (positive and negative) and trained a model capable of computing similarity metrics between new texts and these graphs. Finally, we applied the model to the test set to predict the polarity of unseen texts and thus evaluate its accuracy and generalization capacity.

KEYWORDS: eye-tracking, heatmaps, sentiment, model, prediction

Capítulo 1

Introducción

1.1. Motivación

Las redes sociales han adquirido un papel fundamental en la sociedad, no solo como fuente de información, sino también como un espacio donde las personas expresan sus opiniones y emociones sobre diversos temas. Este volumen masivo de interacciones ha impulsado el desarrollo de la minería de opinión (Nigro, 2020), un campo que permite analizar los sentimientos de los usuarios a partir de sus interacciones en línea.

El análisis de sentimiento se ha convertido en una herramienta clave para empresas y organizaciones, ya que permite medir la percepción pública sobre productos, servicios o eventos en tiempo real. Sin embargo, los métodos tradicionales de análisis de sentimiento suelen basarse exclusivamente en información textual para medir dicha percepción del usuario, dejando de lado otras fuentes de información relevantes, como imágenes, videos, o la interacción visual del usuario con el contenido (Xu et al., 2019).

En este trabajo, buscamos enriquecer el análisis de sentimiento con información obtenida a través del rastreo ocular. El rastreo ocular o *eye-tracking* consiste en medir y analizar la interacción visual del usuario con un elemento, midiendo los movimientos oculares, qué regiones de la pantalla está visualizando y durante cuánto tiempo (Wan Adnan et al., 2013). Ello ayudará a entender mejor cómo los usuarios procesan y perciben la información antes de emitir una opinión, proporcionando mayor interpretabilidad y precisión en la clasificación de la polaridad, o lo que es lo mismo, el sentimiento de una opinión (positivo, negativo o neutral).

Algunas de las ventajas de este enfoque más completo de análisis de sentimiento son:

1. **Mejor comprensión de los modelos de análisis de sentimiento**: Este enfoque permite visualizar e interpretar qué factores influyen en la clasificación del sentimiento, aumentando la transparencia y comprensión del modelo.

2. **Mayor precisión en la clasificación**: Al considerar la atención visual de los usuarios, se obtiene una mejor interpretación del contexto en el que se genera una opinión.

3. **Identificación de los factores que influyen en la polaridad**: Se puede analizar qué partes específicas del contenido afectan más la opinión del usuario.

Algunas de las utilidades de este enfoque pueden ser (Muñoz et al., 2024):

1. **Optimización de páginas web y experiencia del usuario**: Si se puede analizar la experiencia de usuario interactuando con una interfaz, se puede optimizar dicha interfaz para favorecer al máximo la experiencia.

2. **Creación de anuncios publicitarios**: Se pueden crear anuncios publicitarios teniendo en cuenta que elementos son más atractivos para el público.

3. **Investigación de mercados**: Se puede analizar qué partes de un producto gustan más o menos al público. Y también se puede hacer una predicción para futuros productos.

1.2. Objetivos

El objetivo principal de este Trabajo de Fin de Máster (TFM) es proponer un modelo de análisis de sentimiento que combine información textual y visual, con el fin de mejorar tanto la precisión como la interpretabilidad del modelo en la clasificación de polaridad.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

1. **Construcción de un conjunto de datos enriquecido con información visual**:

   - Dado que no existen datasets previos que integren texto con información visual de este tipo (mapas de calor), construir un conjunto de datos con las características apropiadas para este trabajo.
   - Extraer texto a partir de fotogramas que contienen mensajes en redes sociales y sus respuestas asociadas.
   - Asociar cada fotograma con su correspondiente mapa de calor, reflejando la interacción visual del usuario.
   - Integrar la información textual y visual en un conjunto de datos etiquetado con polaridad (positivo/negativo).

2. **Desarrollo de un modelo basado en redes neuronales capaz de analizar sentimiento en un conjunto de datos como el presentado**:

   - Diseñar un modelo que combine redes neuronales y representaciones en forma de grafos para capturar la polaridad textual y visual.
   - Definir métricas que permitan la clasificación de textos en función de su cercanía a representaciones positivas o negativas.
   - Ajustar y optimizar el modelo predictivo para maximizar la precisión.

3. **Evaluación y comparación del modelo propuesto**:

- Medir la precisión, el rendimiento y la interpretabilidad del modelo desarrollado.
- Comparar los resultados frente a modelos preentrenados más complejos.
- Analizar la contribución específica de la información visual en la clasificación de polaridad.

1.3. Estructura del resto del documento

En esta sección se explicará la estructura de la memoria, que se divide en 7 capítulos, siendo el primero la introducción.

En el capítulo 2, se presenta cuál es el estado del arte en el campo de la minería de opinión en Internet, centrándonos en la inclusión de información visual y rastreo ocular. Es decir, se toman y explican trabajos de otros autores que realizaron trabajos similares a lo que nosotros vamos a presentar. Siendo esto así porque algunos de esos trabajos nos han servido como referencia teórica al comenzar con nuestro proyecto, ya que el rastreo ocular en el análisis de sentimiento es una metodología relativamente nueva y en desarrollo.

En el capítulo 3, se explica paso a paso cómo se generan los conjuntos de datos que se van a utilizar, partiendo de los vídeos en los que el usuario escribe una respuesta a un tweet y los mapas de calor, y llegando hasta un conjunto de datos textual enriquecido con toda esta información.

En el capítulo 4, se establece cuál es el modelo propuesto para nuestro TFM. Explicando la estructura del modelo, desde lo más básico hasta su funcionamiento más completo.

En el capítulo 5, se presenta la experimentación realizada para probar nuestro modelo. Básicamente, en este capítulo explicamos cómo llevamos a la práctica nuestro modelo propuesto, cómo medimos el rendimiento del mismo, y con qué otros modelos y/o técnicas lo vamos a comparar para evaluar nuestros resultados.

En el capítulo 6, presentaremos los resultados que hemos obtenido. Y, posteriormente, compararemos los resultados obtenidos con nuestro modelo con los obtenidos con de modelos más complejos.

En el capítulo 7, se presentan las conclusiones finales obtenidas durante la realización del proyecto. Además, se presentan posibles vías para ampliar nuestro TFM actual.

Además, se incluye un anexo donde se presenta un enlace al repositorio *Github* donde se aloja el código empleado en nuestra implementación.

Capítulo 2

Materiales y métodos

En este capítulo se describirán los materiales y métodos utilizados para obtener posteriormente un conjunto de datos mediante el rastreo ocular (*eye-tracking*). Se explica cómo funciona esta herramienta y cómo ha permitido obtener los datos necesarios para dicho conjunto de datos. Por otro lado, también explicaremos algunos trabajos relacionados con el nuestro.

2.1. Eye-tracker

Un *eye-tracker* (o rastreador ocular en español) es un dispositivo o sistema que permite medir y registrar el movimiento de los ojos de una persona. Su objetivo principal es determinar dónde está mirando una persona, durante cuánto tiempo y con qué secuencia de movimientos oculares lo hace.

Los *eye-tracker*s (Wan Adnan et al., 2013) suelen usar tecnologías como cámaras infrarrojas que detectan el reflejo de la luz en la córnea y la retina, o algoritmos de seguimiento ocular que calculan la dirección de la mirada y la fijación visual. Posteriormente, se utiliza un software que procesa los datos generados por estos dispositivos para generar mapas de calor, métricas, etc.

En el caso de nuestro trabajo, se ha utilizado el *eye-tracker* Tobii Pro Nano (Niehorster et al., 2020) con software Tobii Pro Lab para analizar los datos. Es importante mencionar que el dispositivo no es tipo gafas, sino que se coloca en la parte inferior de la pantalla del ordenador, lo que permite una mayor comodidad y naturalidad en la interacción del usuario (ver figura 2.1). El software "Tobii Pro Lab" es una herramienta avanzada que permite no solo la captura y el análisis de los datos de seguimiento ocular, sino también la integración de estos datos con otras fuentes de información, como cuestionarios y datos de comportamiento. Sin embargo, es importante destacar que, aunque el software es muy potente, aún presenta limitaciones y puede no ser capaz de resolver a la perfección completamente todos los aspectos de este TFM. Un ejemplo de estas limitaciones técnicas puede ser la precisión con la que se detecta la región exacta de la pantalla que el usuario está observando en un momento dado, ya que puede haber pequeños retardos temporales o imprecisiones espaciales a la hora de realizar dichas mediciones.

A continuación se muestra una imagen del Eyetracker Tobii Pro Nano, y sus especificaciones técnicas en la figura 2.1 y la tabla 2.1.



Figura 2.1: Eyetracker Tobii Pro Nano (Pro, 2019)

Tabla 2.1: Especificaciones del sistema de seguimiento ocular (Pro, 2019)

Características	**Especificación**
Técnica de seguimiento ocular	Reflexión corneal
Frecuencia de muestreo	60 Hz
Precisión	0.1° RMS (condiciones óptimas)
Exactitud	0.3° (condiciones óptimas)
Seguimiento binocular	Sí
Movimiento libre de cabeza	35 cm × 30 cm a 65 cm
Distancia operativa	45–85 cm
Ángulo máximo de mirada	30°
Tamaño máximo de pantalla	19” (relación 16:9)
Latencia total del sistema	17 ms
Recuperación tras parpadeo	Inmediata (1 fotograma)
Recuperación de mirada	Inmediata (1 fotograma)
Datos por ojo	Timestamp, origen y punto de mirada, diámetro pupilar
Dispositivo	
Dimensiones	17 cm × 1.8 cm × 1.3 cm
Peso	59 g
Procesador	EyeChip
Conectividad	USB 2.0 Tipo A
Sistema operativo compatible	Windows 10 (edición Pro)
Compatibilidad	
Aplicaciones	Tobii Pro Lab, Eye Tracker Manager, SDK de Tobii Pro

2.2. Análisis de sentimiento

Se podría decir que el rastreo ocular tiene aplicación en varios sectores, desde estudios de usabilidad, publicidad y diseño de interfaces, hasta aplicaciones médicas (análisis cognitivos). En nuestro contexto, que es el análisis de sentimiento, el *eye-tracking* ofrece una metodología novedosa en la que ampliamos la información sobre qué opina el usuario al interactuar con un sistema software en una determinada situación (como puede ser leer un texto en una pantalla). En lugar de analizar exclusivamente el contenido verbal o textual, también se observa cómo ese usuario procesa visualmente ese contenido antes de emitir una opinión.

El análisis de sentimiento consiste en detectar información subjetiva en textos e imágenes mediante modelos de aprendizaje automático y procesamiento del lenguaje natural. Y como campo, se encarga de identificar y clasificar el sentimiento u opinión de un usuario respecto a un contenido, normalmente asignando una polaridad (positiva, negativa o neutra). Como decíamos antes, en la mayoría de los trabajos existentes, este análisis se basa únicamente en el contenido textual, y cuando se usa contenido visual, es solo de manera básica y más bien teórica. En nuestro caso, proponemos ir un paso más allá, incorporando información visual adicional mediante mapas de calor generados a partir del rastreo ocular. La idea es que, al observar cómo una persona lee o interactúa visualmente con un contenido, se pueda obtener información adicional sobre su nivel de atención o interés o reacción emocional. Esto permitirá enriquecer el análisis de sentimiento.

Para nuestro trabajo, el *eye-tracking* o rastreo ocular se trata de una tecnología que permite guardar los movimientos oculares de una persona mientras observa una pantalla. En mapas de calor en los que las áreas en las que el usuario ha prestado más atención tienden a un color rojo, y en las que ha prestado menos atención al verde. Esta captura de la interacción del usuario se realiza mediante sensores ópticos, combinados con cámaras infrarrojas, que permiten detectar las coordenadas exactas en las que el usuario está mirando, y durante qué lapso de tiempo. De esa manera, se puede entender en qué partes se ha fijado el usuario y en qué grado.

2.3. Trabajos relacionados

El uso del *eye-tracking* en tareas de reconocimiento de emociones ha sido explorado en numerosos estudios. En la revisión sistemática realizada por Lim et al. (2020), se analiza un conjunto de 11 trabajos que aplican técnicas de rastreo ocular para la clasificación de emociones, destacando las características más utilizadas y los clasificadores aplicados.

A continuación, se resume la información clave extraída de dicho estudio:

- El mayor nivel de precisión (90 %) se obtuvo utilizando redes neuronales artificiales como clasificador, empleando una combinación de tamaño de pupila, posición de la pupila y velocidad del movimiento ocular como características de entrada. Tal y como se recoge en el artículo Raudonis et al. (2013).

- Los estudios que reportaron mejores resultados (superiores al 85 % de precisión) utilizaron al menos tres características combinadas Lu et al. (2015), Wang et al. (2018) y Tsang (2018).
- Los enfoques menos exitosos se basaban exclusivamente en el diámetro pupilar, obteniendo precisiones bajas, en torno al 59 % Oliva and Anikin (2018) y Zheng et al. (2014).
- Las características más utilizadas fueron:
  - Diámetro de pupila (8 estudios).
  - Duración de las fijaciones (4 estudios).
  - Posición de la pupila (2 estudios).
  - Señales EOG (electrooculografía) (2 estudios).
- Solo uno de los estudios analizados reportó la velocidad del reconocimiento emocional, alcanzando resultados de clasificación en menos de 2 segundos desde la presentación del estímulo emocional (Raudonis et al., 2013).

Características utilizadas	**Número de estudios**	**Precisión media**
Tamaño de pupila	8	∼60–85 %
Duración de fijaciones	4	∼75–88 %
Posición de pupila	2	Hasta 90 % (combinado)
Velocidad ocular	1	Hasta 90 % (combinado)
EOG	2	Variable

Tabla 2.2: Resumen de características extraídas de estudios con *eye-tracking* para clasificación emocional, según Lim et al. (2020).

Este análisis, cuyos resultados acabamos de ver en la tabla 2.2, refuerza la idea de que la combinación de múltiples fuentes de información visual puede mejorar considerablemente el rendimiento en tareas de clasificación emocional o afectiva, lo cual es coherente con el enfoque propuesto en este trabajo, donde se integra el contenido textual con mapas de atención visual.

Capítulo 3

Generación del conjunto de datos

En este capítulo, se detallará el procedimiento seguido para la generación del conjunto de datos, desde la captura inicial de datos hasta el procesamiento final de los fotogramas y mapas de calor.

3.1. Captura de datos

Vamos a partir de un conjunto de tweets y sus respuestas, junto con vídeos que capturan la interacción del usuario al escribir dichas respuestas. Para generar todo esto, se midió la interacción visual de un grupo de 11 usuarios (que son alumnos del Máster en Ingeniería Informática) mientras respondían a 15 tweets de la red social X o twitter, que trataban temas esencialmente gastronómicos.

Con esos 15 tweets, se pidió a los 11 usuarios que respondieran a cada uno de ellos. A cada usuario se le presentó el contenido del tweet en pantalla, y se almacenó, por un lado, un vídeo que mostraba la pantalla durante el proceso de la respuesta, y, por otro lado, los mapas de calor asociados al rastreo ocular, mostrando la interacción visual del usuario durante su respuesta.

Entonces, el conjunto de datos inicial está compuesto por un conjunto de 15 tweets, y cada uno de los cuales tiene asociadas 11 respuestas, generando un total de:

- 165 vídeos, donde se graba la pantalla mientras un usuario escribe su respuesta.
- 165 mapas de calor generados mediante *eye-tracking*, los cuales reflejan las zonas de mayor atención visual durante la escritura de la respuesta.

Cabe destacar que los vídeos capturan únicamente la pantalla del dispositivo del usuario, mostrando el proceso de lectura del tweet y la redacción de la respuesta, sin grabar al usuario en sí.

3.2. Extracción de fotogramas clave

Dado que los vídeos contienen información redundante y ocupan un espacio considerable, no resulta viable utilizarlos directamente en el análisis. En nuestro caso, solo nos interesa el fotograma en el que la respuesta ya ha sido escrita por completo, ya

que este será el punto de referencia para superponer la información del texto con los mapas de calor.

Por lo tanto, se extrae de cada vídeo un único fotograma clave que representa la interacción final del usuario con el tweet y su respuesta. El resto de los fotogramas del vídeo son descartados para reducir la cantidad de datos innecesarios y optimizar el procesamiento.

3.3. Normalización y recorte de imágenes

Una vez extraídos los fotogramas y los mapas de calor, nos encontramos con dos problemas principales:

Primero, tenemos diferencias en el tamaño de las imágenes, en tanto que los fotogramas extraídos tienen una resolución de 1440x810 píxeles, mientras que los mapas de calor de 1920x1080. Entonces, como queremos ser capaces de superponer ambas imágenes, reescalamos los fotogramas a 1920x1080.

Segundo, ahora nos encontramos que, aunque las imágenes y los mapas de calor tienen el mismo tamaño, gran parte de los píxeles no presentan información útil, ya que en el caso de los fotogramas, el tweet y la respuesta siempre están en la misma área de la pantalla, y en los mapas de calor pasa algo similar. Haciendo que las esquinas de la pantalla sean poco relevantes para nuestro estudio. Entonces, reducimos tanto fotogramas como mapas de calor a un tamaño adecuado de 600x450, en la que tenemos la información necesaria.

Entonces, podría decirse que el proceso permite obtener imágenes normalizadas y más eficientes para el análisis posterior, asegurando que la información relevante del tweet y la respuesta se mantenga intacta.

Un ejemplo de un fotograma y su mapa de calor asociado, podría ser el mostrado en la figuras 3.1 y 3.2.



Figura 3.1: Ejemplo de un fotograma con un tweet y su respuesta

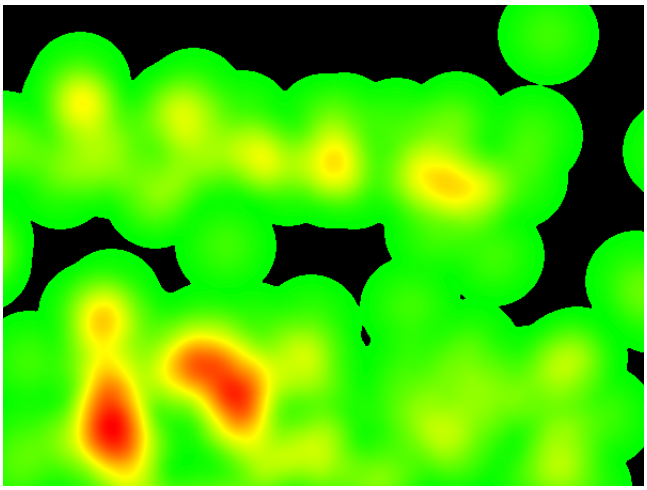


Figura 3.2: Ejemplo del mapa de calor asociado

3.4. Generación del conjunto de datos textual a partir de información visual

Una vez se han obtenido los 165 fotogramas y mapas de calor asociados a cada pareja tweet-respuesta, el siguiente paso es la extracción del texto presente en los fotogramas, potenciado con la información de rastreo ocular presente en los mapas de calor.

3.4.1. Extracción del texto

El primer paso es extraer el texto tanto de los tweets como de las respuestas que están en el fotograma correspondiente. Para ello, se ha pensado dos enfoques distintos, dependiente de si hablamos de los 15 tweets originales o de las respuestas, la diferenciación se hace de la siguiente forma:

- **Tweets originales:** En este caso su contenido textual va a ser siempre el mismo, siempre vamos a tener la misma lista de palabras en las mismas coordenadas dentro del mismo fotograma. Entonces, podemos ahorrarnos la extracción de texto per se, y almacenar las palabras y las coordenadas de manera estática para cada uno de los 15 tweets originales.

- **Respuestas:** Este caso es diferente al anterior, no sabemos qué texto podemos esperarnos, por lo que para esto se aplicará extracción de texto en imágenes mediante una herramienta de reconocimiento óptico de caracteres (OCR o *Optical Character Recognition*), en nuestro caso, mediante la librería `pytesseract` (Jayoma et al., 2020). Esto permite identificar tanto las palabras como sus coordenadas en los fotogramas.

Una vez tenemos localizadas las palabras, se dibujan cajas delimitadoras tanto en los fotogramas como en los mapas de calor, facilitando la verificación visual del proceso. Un ejemplo de ello sería el mostrado en las figuras 3.3 y 3.4:

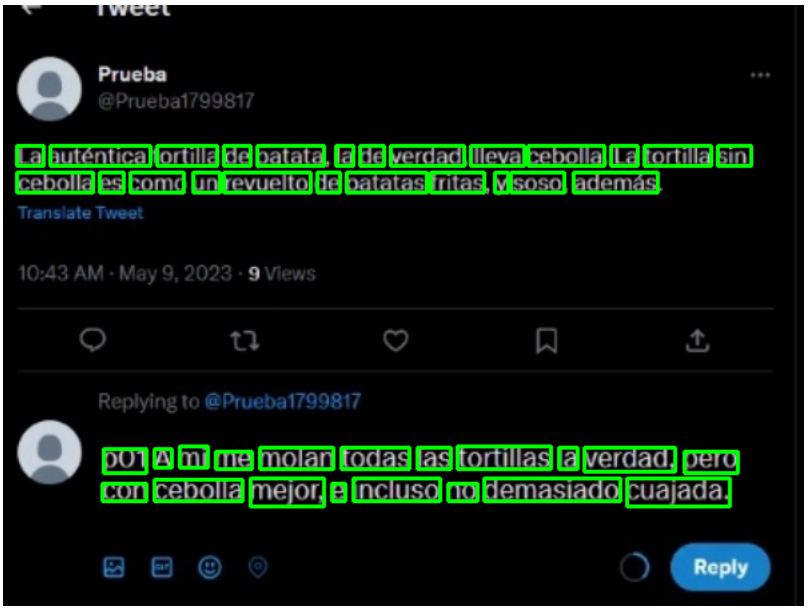


Figura 3.3: Ejemplo de un fotograma al que se le ha aplicado OCR

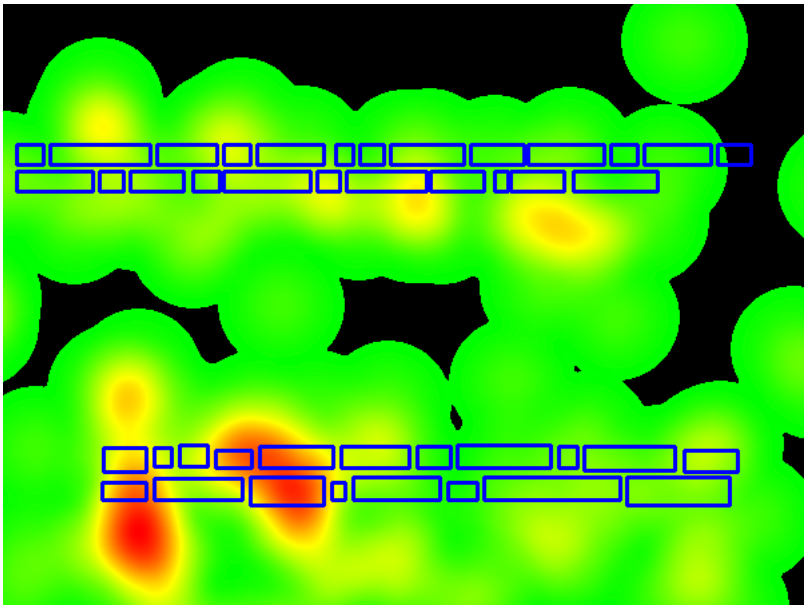


Figura 3.4: Coordenadas OCR superpuestas en el mapa de calor

3.4.2. Cálculo del score de atención

Ahora que tenemos para cada tweet y para cada respuesta el contenido textual, la idea es, a partir de la información de los mapas de calor, extraer un *score* que refleja el nivel de atención visual en torno a cada palabra. Para ello, se calcula un *score* basado en el promedio del componente de tono (Hue) (Contributors, 2024) en el área de cada caja dentro del mapa de calor. Este valor se normaliza entre 0 y 1, donde valores más altos indican mayor atención. El siguiente procedimiento se describe en el algoritmo 1.

Algorithm 1 Cálculo del *score* de un pixel basado en el HUE (Contributors, 2024)

Entrada: Valores de color RGB r, g, b en el rango [0, 255]
Salida: *Score* basado en el valor de tono H
$r \leftarrow r/255$
$g \leftarrow g/255$
$b \leftarrow b/255$
$M \leftarrow \text{máx}(r, g, b)$
$m \leftarrow \text{mín}(r, g, b)$
$C \leftarrow M - m$
$H \leftarrow 0$
if $M = r$ **then**
  $H \leftarrow 60 \times \left(\frac{g-b}{C} \,\%6\right)$
else if $M = g$ **then**
  $H \leftarrow 60 \times \left(2 + \frac{b-r}{C}\right)$
else if $M = b$ **then**
  $H \leftarrow 60 \times \left(4 + \frac{r-g}{C}\right)$
end if
if $H < 0$ **then**
  $H \leftarrow 0$
end if
if $H > 120$ **then**
  $H \leftarrow 120$
end if
$score \leftarrow 1 - \frac{H}{120}$
return *score*

Además del *score* relativo a cada palabra, también se calcularán *scores* extendidos, que consisten en expandir verticalmente el área original de la caja a partir de la cual se calcula el *score* promedio. Esto se hace mediante un parámetro de expansión b, generando distintos valores como:

- `score`: sin expansión.
- `scoreb05`, `scoreb1`, `scoreb2`, ..., `scoreb4`: expansión progresiva del área a medir en el fotograma. Por ejemplo, para *scoreb1*, se tomaría el área de la caja original correspondiente a una palabra, y posteriormente habría que sumar la altura*1 arriba y abajo. Con b=2 la altura*2 arriba y abajo, y así sucesivamente.
- `score_extendido`: expansión máxima ($b = 100$), limitada por los márgenes verticales de la imagen.

La idea de esta expansión es enriquecer aún más la información relativa a cada palabra, no solo teniendo el *score* de su región, sino también un *score* que contemple regiones verticales próximas, con la intención de que esto pueda arrojar más información sobre el contexto de la palabra y de los focos de atención en la zona.

3.4.3. Contenido del fichero CSV resultante

Ahora que tenemos el conjunto de datos textual combinado con los *scores* calculados mediante la información visual, tenemos que para cada texto, cada palabra detectada (por OCR o por coordenadas fijas) se registra como una tupla en la que se almacena no solo el contenido de la palabra, sino los *scores* correspondientes. La manera de almacenar el conjunto de datos es un fichero CSV con el siguiente contenido:

- `num_tweet`: identificador del tweet original.
- `palabra`: palabra detectada.
- `respuesta`: 0 si la palabra pertenece tweet original, 1 si a la respuesta.
- `score`, `scoreb05`, ..., `scoreb4`, `score_extendido`: niveles expansión del score.
- `polaridad` : 0 negativo y 1 positivo

3.4.4. Dos análisis posibles

Para este trabajo, y siguiendo este proceso de generación del conjunto de datos, vamos a realizar dos análisis de sentimiento distintos, lo que se traduce en dos conjuntos de datos distintos. El proceso de generación visto en este capítulo será común para ambos casos, la diferencia radicará en la polaridad o etiqueta que se le asignará a cada tweet.

En el primer análisis, asignaremos polaridad a las respuestas con base en su opinión sobre el tweet al que responden. Es decir, si están a favor polaridad positiva, y si están en contra negativa. Entonces, lo que queremos medir en este caso es si los usuarios están de acuerdo o en desacuerdo con los 15 tweets originales

En el segundo análisis, se cogerán todos los tweets y todas las respuestas como un conjunto único donde no se distingue entre tweets y respuestas, es decirm cada texto es independiente de los demás. A partir de ahí se hará un análisis de sentimiento puro. Es decir, en este caso, se busca hacer un análisis de sentimiento más clásico en el que cada texto es independiente de los demás.

Capítulo 4

Metodología

En este capítulo se va a establecer la estructura así como el funcionamiento del modelo de análisis de sentimiento propuesto.

Dividiremos la explicación de nuestro modelo en tres partes: las dos primeras (preprocesamiento y construcción de grafos de polaridad) serán los preliminares necesarios para generar las entradas de nuestro modelo, mientras que la tercera (ajuste del modelo) servirá para explicar qué tipo de modelo predictor vamos a utilizar y cómo se realiza el proceso de entrenamiento y de test del mismo.

4.1. Preprocesamiento

En esta primera fase leeremos los textos a estudio desde el conjunto de datos etiquetado en formato CSV que hemos obtenido en el capítulo anterior. Lo primero será cargar el conjunto de datos: Tendremos una lista de tweets; y cada tweet tendrá tanto su polaridad como una lista de palabras; y cada palabra será una tupla que contendrá (palabra, *scoreb05*, *scoreb1*, *scoreb2*, *scoreb3*, *scoreb4*, *score_extendido*).

En esta primera fase de preprocesamiento, vamos a dejar los *scores* de lado y nos vamos a centrar únicamente en la información textual. Más concretamente, vamos a realizar un preprocesamiento para lograr un formato compacto de los textos.

Este proceso seguirá los siguientes pasos, para cada tweet a preprocesar. Ponemos todas las palabras en minúsculas, borramos los caracteres no convencionales (emojis), así como las etiquetas/hashtags (#), las menciones a otros usuarios (@) y puntos suspensivos y demás signos de puntuación.
Posteriormente, eliminamos palabras neutras que no aporten mucho al análisis de sentimiento, como pueden ser los artículos o las preposiciones.

Para poder realizar todo esto, simplemente hemos ido recorriendo la lista de textos, y la lista de palabras para cada uno, aplicando en el proceso las distintas modificaciones pertinentes. Para ello, se utilizan distintas funciones propias de Python, como *str()* para convertir un texto a string, *lower()* para convertirlo a minúsculas, así como librerías como `re`, utilizada para poder manipular expresiones regulares que nos ayudan a modificar con facilidad un texto, eliminando enlaces, o caracteres especiales.

4.2. Construcción de los grafos de polaridad

Una vez todos los textos han sido preprocesados, debemos generar los grafos de polaridad. Para ello seguiremos los siguientes pasos.

Primero, se realiza la división train/test del conjunto de datos, separando los datos que se utilizarán para entrenar el modelo y los que se utilizarán para evaluar su rendimiento. A continuación, se divide el conjunto de entrenamiento en textos negativos y positivos, clasificando así los textos según su polaridad.

Luego, se construye un grafo de polaridad negativo con los textos de entrenamiento negativo y un grafo de polaridad positivo con los textos de entrenamiento positivos.

Para lograr esto, cada texto se representa como un grafo, donde cada palabra del texto se convierte en un nodo del grafo. La construcción del grafo de polaridad implica componer todos los pequeños grafos que representan los textos individuales, creando un grafo más grande que captura la polaridad general de los textos en el conjunto de entrenamiento.

En estos grafos que se van construyendo, además de incluir los textos como es lógico, también se incluirá la información recogida en los *scores*, tanto a nivel de nodo como a nivel de arista. Primero, a nivel de nodo, el peso de dicho nodo (palabra), será una suma ponderada del *score* normal y de uno de los *scores* extendidos (b05,b1,b2,b3,b4 o extendido) que hayamos elegido. Esto es así ya que nunca utilizaremos todos los *scores* a la vez para asignar el peso a los nodos, sino que solo usaremos el normal y uno de los cinco tipos de *score* extendido. Por otro lado, las aristas también contendrán un peso, y este será el promedio de los pesos de los dos nodos que la componen.

Para la creación de los grafos de polaridad se ha optado por la elaboración de una función propia *build graph* que explicaremos ahora mediante un pseudocódigo descrito en el algoritmo 2.

Algorithm 2 build graph (distancia, textos, alpha, beta)

Require: $distancia \geq 0$

```
# Lista vacía de grafos
grafos ← ()
for texto in textos do
    # Conjuntos vacíos de nodos y aristas
    V ← {}
    E ← {}
    F ← (V, E)
    for palabra in texto do
        score ← alpha * scoreNormal(palabra) + beta * scoreExtendido(palabra)
        V ← V ∪ {palabra}
        V[palabra][peso] ← score
    end for
    # Se extraen los nodos del conjunto como una lista
    nodos ← V
    for i ← 0 hasta i ← longitud(nodos) do
        for j ← i + 1 hasta j ← i + distancia do
            E ← E ∪ {(nodos(i), nodos(j))}
        end for
    end for
    grafos ← grafos ∪ F
end for
G ← composicion(grafos)
V' ← G(nodos)
for E' in G(aristas) do
    u, v ← E'
    E'[peso] ← (V'[u][peso] + V'[v][peso]) / 2
end for
return G
```

Como se puede ver, la idea principal es crear un grafo para cada texto, asignando un peso basado en los *scores* a cada nodo y posteriormente, componer todos los diferentes grafos para crear el grafo de polaridad resultante. En dicho grafo resultante, asignar el peso también a las aristas. Sin embargo, hay un concepto muy importante que no hemos introducido previamente, que es el de la distancia.

Partiendo de una lista de los nodos de un grafo, la distancia es el número de nodos posteriores con los que un nodo n tendrá conexión. Por ejemplo, para distancia uno, un nodo iría conectado con el siguiente, para distancia dos, con los dos siguientes, etc.

Por ejemplo, veamos cómo se representaría el texto "Mañana jueves lloverá mucho", con distancias 1,2 y 3.



Figura 4.1: Frase *Mañana jueves lloverá mucho* con distancia = 1

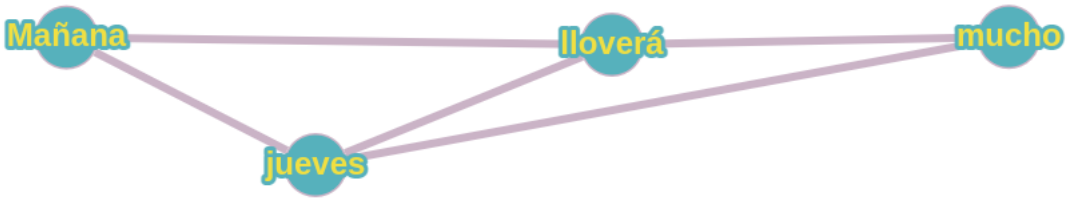


Figura 4.2: Frase *Mañana jueves lloverá mucho* con distancia = 2

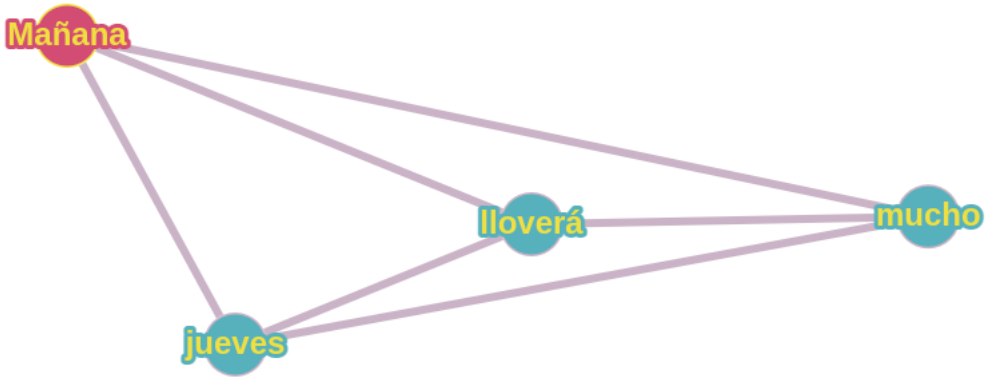


Figura 4.3: Frase *Mañana jueves lloverá mucho* con distancia = 3

En las figuras 4.1, 4.2 y 4.3, creadas con la herramienta llamada Graph Online (admin@graphonline.ru, 2015) podemos ver cómo se representa la frase *Mañana jueves lloverá mucho* empleando distancias 1, 2 y 3 respectivamente. De esta forma podemos ver de manera gráfica cómo se cumple que en el caso de distancia 1, cada nodo está unido al siguiente, cómo en el caso de distancia 2 está unido a los dos siguientes y cómo en el caso de distancia 3 está unido a los 3 siguientes. Dicho esto, podemos ver que cuanto más aumentemos la distancia, más conexos estarán los grafos que creemos, tanto los asociados a un texto como los grafos de polaridad. Sin embargo, eso trae consigo un aumento de la complejidad a la hora de calcular las métricas de centralidad, por lo que nuestro predictor puede mejorar o empeorar según la distancia utilizada. Y también parece claro que cuanto mayor sea la distancia, mayor será el tiempo de cómputo requerido para la creación de los grafos y el cómputo de dichas métricas. En nuestro caso, como estamos trabajando con un conjunto de datos pequeño (165 tweets), cuyos textos son cortos en su mayoría, hemos decidido utilizar distancia 1, y representar cada pequeño tweet como un grafo línea.

4.3. Ajuste del modelo de red neuronal

En esta última fase, explicaremos el proceso de entrenamiento y test del modelo, así como la arquitectura y el funcionamiento del mismo.
Una vez tenemos ambos grafos de polaridad, los usaremos como medio de comparación y obtendremos las métricas de centralidad para comparar los grafos de cada texto con los grafos de polaridad.
Para ello obtendremos las métricas de *eigenvectors*, *betweenness*, *nodos en común* y *aristas en común*. Y dado que tenemos dos grafos de polaridad, cada texto se verá representado por 8 métricas. La forma de obtener dichas medidas se explica en el algoritmo 3.

Algorithm 3 Obtención de las métricas de centralidad (distancia, alpha, beta)

$X_TRAIN \leftarrow []$ Lista donde se almacenan las métricas para el conjunto de entrenamiento.
$G \leftarrow (V, E)$ donde V es el conjunto de vértices del grafo y E es el conjunto de aristas del mismo.
for `texto in conjunto_train` **do**
  $G \leftarrow$ build_graph(distancia, texto, alpha, beta)
  $m1 \leftarrow$ eigenvector(G , grafo de polaridad negativo)
  $m2 \leftarrow$ eigenvector(G , grafo de polaridad positivo)
  $m3 \leftarrow$ betweenness(G , grafo de polaridad negativo)
  $m4 \leftarrow$ betweenness(G , grafo de polaridad positivo)
  $m5 \leftarrow$ nodos en común(G , grafo de polaridad negativo)
  $m6 \leftarrow$ nodos en común(G , grafo de polaridad positivo)
  $m7 \leftarrow$ aristas en común(G , grafo de polaridad negativo)
  $m8 \leftarrow$ aristas en común(G , grafo de polaridad positivo)
  $X_TRAIN \leftarrow X_TRAIN \cup (m1, m2, m3, m4, m5, m6, m7, m8)$
end for

Antes de seguir explicando el proceso de entrenamiento, vamos a explicar lo que significa cada métrica:

- eigenvector : "Esta métrica evalúa las relaciones de los nodos con un esquema de puntuaciones. Los nodos de alta puntuación representan a aquellos vértices que tienen mayores conexiones y, por lo tanto, poseen un nivel de relevancia superior" (Van Mieghem, 2014)
- betweenness : "El betweenness, o centralidad, entre aristas se define como el número de caminos más cortos que pasan por una arista de un grafo. A cada arista de la red se le puede asociar un valor de centralidad entre aristas" (Lu and Zhang, 2013)
- nodos en común : Número de nodos en común que tiene un grafo de polaridad con el grafo asociado al texto en cuestión.
- aristas en común : Número de aristas en común que tiene un grafo de polaridad con el grafo asociado al texto en cuestión.

Es importante mencionar que, para estas métricas, también se tendrá en cuenta el *score* de los nodos/aristas involucrados. Por ejemplo, para el caso de nodos o aristas

en común, en vez de sumar +1 cada vez que se encuentra una coincidencia, se suma (1 * diferencia de pesos entre los nodos/aristas en el grafo del tweet y en el de polaridad). Esto es así porque no solo tenemos que valorar que un nodo o una arista del grafo del tweet esté en alguno de los grafos de polaridad, sino que también habrá que valorar que tengan pesos similares, premiando los casos en los que así sea, y penalizando aquellos en los que haya más diferencias. Con el betweenness y el eigenvector, también se sigue la misma lógica, premiando aquellos casos con *scores* más similares.

Una vez tenemos el array con las 8 métricas para representar un texto, introducimos dicho array al conjunto *X_train* de nuestro modelo (algoritmo 3), mientras que el conjunto *Y_train* serán las etiquetas conocidas para esos textos, ya que como habíamos explicado antes, el conjunto de datos es etiquetado.

El modelo en cuestión será una red neuronal (perceptrón multicapa). Dicha red neuronal seguirá una arquitectura de 5 capas con 32, 128, 128, 128, 32 y 8 neuronas respectivamente y con el parámetro *solver = "adam"*. Se utilizó ese *solver*, ya que parece ser el que más favorece el entrenamiento para conjuntos de datos pequeños como serán los nuestros.

Por último, con los textos del conjunto test, seguiremos el mismo proceso que anteriormente, crearemos grafos para cada texto. Tras ello, compararemos dichos grafos con ambos grafos de polaridad y tendremos así las 8 métricas para cada uno, y pasaremos dicho array al modelo para posteriormente predecir su polaridad. Después de eso obtendremos un nuevo array llamado *predicciones* que devolverá para cada texto un valor de 0 si el modelo ha decidido que el texto sea negativo y un 1 si el modelo decide que sea positivo. Dichas predicciones podremos compararlas con las etiquetas reales para conocer el rendimiento de nuestro modelo tal y como veremos más adelante.

Capítulo 5

Experimentación

En este capítulo se aborda la fase de experimentación, en la que se pone en práctica el modelo descrito en el capítulo anterior. Se explicarán los conjuntos de datos utilizados, su lectura, la evaluación del modelo, el uso de las métricas empleadas como referencia para evaluar su rendimiento y una comparación con modelos externos.

5.1. Estructura del conjunto de datos

Tal como se expuso en el capítulo 3, se utilizarán dos conjuntos de datos distintos, debido a que se han planteado dos enfoques de análisis: uno en el que se evalúa la opinión de las respuestas respecto a los tweets originales (análisis relacional), y otro en el que se realiza un análisis de sentimiento puro considerando cada tweet como una unidad independiente. A continuación, se explicará cómo obtener dichos conjuntos y su estructura.

Una vez descritos los conjuntos de datos, se detalla el primer paso del proceso experimental: la lectura de los datos y la evaluación del modelo. Tal como se indicó anteriormente, los conjuntos de datos contienen, entre otros campos, el texto, la polaridad, el *score* normal y varios *scores* extendidos.

Para ambos conjuntos de datos, el procedimiento será idéntico, salvo por la interpretación de la polaridad, ya que cada análisis evalúa un aspecto diferente. Se dividirá el conjunto de datos en 15 bloques, uno por cada tweet original, y dentro de cada bloque se incluirán sus 11 respuestas.

Se generarán tres versiones del conjunto de datos a partir de esta división, simulando una validación cruzada de 3 folds. Los resultados del modelo se obtendrán mediante el promedio del rendimiento en estos tres folds. Esto es especialmente relevante porque, al variar la distribución de tweets en cada fold, también cambiarán los grafos de polaridad que son clave para el entrenamiento del modelo, lo que nos permitirá evaluar su comportamiento ante distintos escenarios de entrada.

Una vez establecido que trabajaremos con tres versiones del mismo conjunto, resta explicar cómo se genera cada fold.
El proceso es sencillo, ya que el número de bloques (15) es divisible entre el número de folds (3). Como es habitual, se necesita una partición en datos de entrenamiento

y de prueba. En cada fold, se utilizarán 10 bloques para entrenamiento y 5 para prueba, lo cual ofrece un equilibrio adecuado. Si se emplearan menos bloques para test, se correría el riesgo de sobreajuste (*overfitting*); en cambio, si se usaran más bloques para test, la parte de entrenamiento podría quedarse escasa, dificultando el aprendizaje del modelo.
La división de los folds será la siguiente:

- **Fold 1:** bloques 1–10 para entrenamiento, bloques 11–15 para test.
- **Fold 2:** bloques 6–15 para entrenamiento, bloques 1–5 para test.
- **Fold 3:** bloques 1–5 y 11–15 para entrenamiento, bloques 6–10 para test.

El resultado final para cada uno de los dos conjuntos de datos serán tres carpetas (una por fold), y en cada una habrá un fichero `train.csv` con los tweets de entrenamiento y un fichero `test.csv` con los tweets de prueba.

En las siguientes imágenes (figuras 5.1, 5.2 y 5.3) se puede apreciar la estructura del conjunto de datos resultante

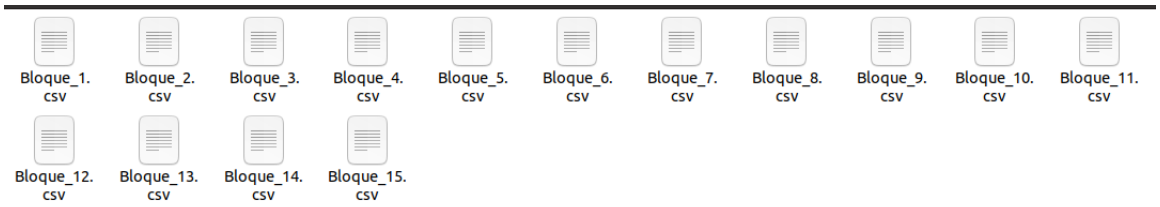


Figura 5.1: Resultado de dividir el conjunto de datos inicial en 15 bloques



Figura 5.2: Resultado de dividir los 15 bloques en 3 folds



Figura 5.3: Cada fold tiene a su vez un conjunto de entrenamiento y de test

5.2. Distribución de los datos

A continuación, se repasa la estructura interna de los conjuntos de datos una vez preparados, observando su distribución interna. Lo primero será ver la distribución del número de palabras para ambos conjunto de datos, que será la misma, ya que el

conjunto de datos es común, lo que cambia son las etiquetas.

Primero veamos en la figura 5.4 la distribución del número de palabras de los 15 tweets originales.

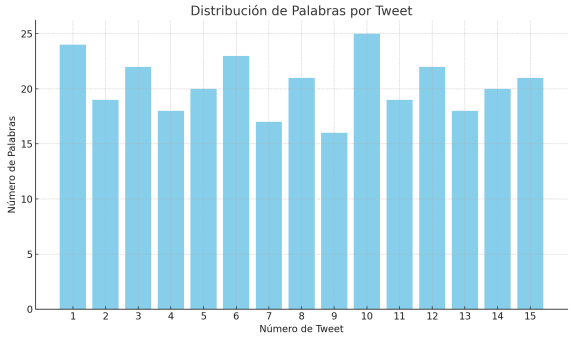


Figura 5.4: Número de palabras para los 15 tweets originales

Ahora, veremos en la figura 5.5 la distribución para los tweets de respuesta, pero aquí es necesario mencionar que se trata de los tweets pasados por la detección de texto OCR, que es una estimación, y que no tiene por qué sacar todas las palabras del fotograma pasado.

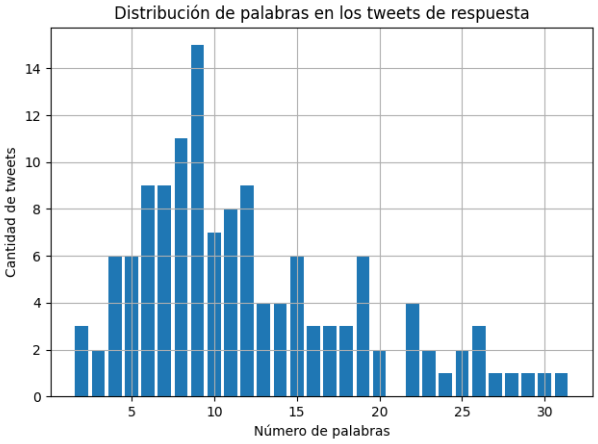


Figura 5.5: Distribución de palabras para las respuestas

Una vez visto esto, pasaremos a analizar cada uno de los dos conjuntos de datos y sus 3 folds.

5.2.1. Conjunto de datos del análisis basado en respuestas

Este primer conjunto de datos aplica la polaridad de manera relacional con los tweets originales. Se asigna valor negativo si la respuesta está en desacuerdo con el tweet original, y positivo si está a favor. Y la distribución de polaridad los folds es la siguiente descrita en la tabla 5.1:

Fold	Train +	Train -	Test +	Test -
Fold 1	41	54	19	26
Fold 2	39	54	21	26
Fold 3	38	54	22	26

Tabla 5.1: Distribución de ejemplos positivos y negativos en entrenamiento y test para cada fold.

5.2.2. Conjunto de datos del análisis de sentimiento puro

En este otro conjunto de datos se trata cada tweet de forma independiente, sin tener en cuenta el tweet original. La polaridad se asigna en función del sentimiento general. La distribución de los folds es la siguiente descrita en la tabla 5.2:

Fold	Train +	Train -	Test +	Test -
Fold 1	46	50	24	24
Fold 2	47	48	23	26
Fold 3	42	52	28	22

Tabla 5.2: Distribución de ejemplos positivos y negativos en entrenamiento y test para cada fold.

5.3. Cálculo de métricas de rendimiento

Tras entrenar y probar el modelo con los datos correspondientes, se calcularán diversas métricas para evaluar su capacidad de clasificación. Las métricas empleadas serán: *AUC*, *Accuracy*, *Precision*, *Recall* y *F1-Score*, definidas como sigue (Pedregosa et al. (2011)):

- Área bajo la curva (AUC) = es una herramienta estadística que se utiliza para medir el acierto en la predicción de eventos binarios
- $Accuracy = \frac{número\ de\ predicciones\ correctas}{número\ de\ predicciones\ totales}$
- Precision para la clase negativa y positiva. $Precision = \frac{True\ Positive}{True\ Positive\ +\ False\ Positive}$
- Recall. $Recall = \frac{True\ Positive}{True\ Positive\ +\ False\ Negative}$
- F1-Score. $F1 - SCORE = \frac{2 * Precision * Recall}{Precision + Recall}$

5.4. Selección de parámetros y estudio de ablación

Como se comentó anteriormente, los grafos de polaridad incorporan un concepto de *score*, que asigna peso a cada nodo, y, por extensión, a cada arista (como promedio de los *scores* de sus nodos extremos).

El *score* es una combinación del *score normal* y de uno de los scores extendidos. Existen seis tipos de scores extendidos: *score_b05*, *score_b1*, *score_b2*, *score_b3*, *score_b4*, y *score_extendido*.
La primera fase del estudio de ablación consiste en seleccionar cuál de estos *scores* extendidos emplear. La segunda fase consiste en determinar el peso relativo entre el *score* normal y el *score* extendido. Para ello se utilizará una suma ponderada de ambos *scores* de la siguiente forma: $\alpha \cdot score + \beta \cdot score_extendido$ donde $\alpha \in [0,0{,}1,0{,}2,\ldots,1]$ y $\beta = 1 - \alpha$. Se probarán así 11 combinaciones diferentes. Por lo que al final, estudiaremos las mejores combinaciones de *score* normal y extendido, así como la mejor combinación de alfa y beta para cada caso

5.5. Comparación con modelos de lenguaje BERT

Finalmente, se comparará el rendimiento del modelo propuesto con el de modelos más complejos, concretamente con variantes de *BERT*. Estos modelos se explicarán con mayor detalle en el capítulo 6, dedicado a la presentación de resultados.
De momento solo comentaremos que los modelos BERT (Tensorflow, 2022) utilizados serán:

- `small_bert/bert_en_uncased_L2_H128_A2` (Turc et al., 2019)
- `small_bert/bert_en_uncased_L4_H128_A2` (Turc et al., 2019)
- `small_bert/bert_en_uncased_L2_H256_A4` (Turc et al., 2019)
- `electra_base` (Turc et al., 2019)

Aquí es importante mencionar que estos modelos BERT son preentrenados. Es decir, no podrán incorporar la información visual que hemos obtenido nosotros mediante los fotogramas y los mapas de calor, por lo que estos modelos tratarán solo con la información textual. Entonces, esta prueba será un buen ejercicio para comprobar que efectivamente la inclusión de las técnicas de rastreo ocular y el enriquecimiento del conjunto de datos con esta información visual aporta a mejorar la precisión del modelo.

Capítulo 6

Resultados

Esta sección la vamos a dividir en 3 partes: Primero, se presentará la forma en la que recogemos y presentamos los resultados obtenidos por nuestro modelo. Posteriormente, se hará la selección de los mejores parámetros y el estudio de ablación pertinente. Y por último, una vez obtenidos los resultados de todas las pruebas, se hará una comparación de nuestros resultados con los obtenidos por otros modelos más sofísticados, utilizando siempre los mismos conjuntos de datos para todos los casos de prueba.

6.1. Presentación de los resultados con nuestro modelo

Para evaluar el rendimiento de nuestro modelo, tal y como se explicó previamente, se llevaron a cabo dos tipos de análisis: uno basado exclusivamente en análisis de sentimiento, y otro a partir de las respuestas al tweet original. En ambos casos, el valor final de cada métrica se ha calculado como la media de los resultados obtenidos en las tres ejecuciones correspondientes a los tres folds utilizados.

En cada uno de estos dos escenarios, se explorarán múltiples combinaciones entre el *score* normal y diversas variantes de *scores* extendidos, moduladas mediante coeficientes de ponderación α y β. Esta da lugar a un total de seis configuraciones diferentes, ya que tenemos seis tipos de scores extendidos:

- Combinación con `score_b05`
- Combinación con `score_b1`
- Combinación con `score_b2`
- Combinación con `score_b3`
- Combinación con `score_b4`
- Combinación con `score_extendido`

Cada combinación da lugar a un fichero de resultados con una estructura similar a la siguiente (en este caso, correspondiente al fichero *score_b05*). Un ejemplo de este resultado se puede ver a continuación en la tabla 6.1.

Tabla 6.1: Ejemplo de métricas resultantes para diferentes combinaciones de Alpha y Beta

Alpha	**Beta**	**Métrica**	**Valor**
0.0	1.0	AUC	0.5982832956517168
		acc	0.5578605200945627
		pred_neg	0.5647319880729491
		pred_pos	0.4682539682539682
		rec_neg	0.923076923076923
		rec_pos	0.10119996962102225
		f1_neg	0.6996362433862434
		f1_pos	0.1581196581196581
0.1	0.9	AUC	0.6005690216216532
		acc	0.5723601260835304
		pred_neg	0.5744131391249693
		pred_pos	0.5416666666666666
		rec_neg	0.923076923076923
		rec_pos	0.13461684514316094
		f1_neg	0.7067517828387394
		f1_pos	0.20456790123456792
...	...	-	-
1.0	0.0	-	-

Esta misma estructura se repite para los ficheros correspondientes a las combinaciones `b1`, `b2`, `b3`, `b4` y *score_extendido*.

Con el objetivo de representar de forma clara la evolución de cada métrica en función del tipo de combinación utilizada, hemos generado gráficas para cada métrica. Cada una de estas gráficas muestra el mejor valor alcanzado por cada combinación *score_normal* - *score_extendido*, así como los valores concretos de (α, β) que permitieron alcanzar dicho rendimiento. Esto permite interpretar visualmente el impacto que tiene cada tipo de *score* extendido en las métricas de rendimiento (por ejemplo, una mejora en el *recall* positivo a costa de una menor precisión en clase negativa, una buena AUC pero una mala precisión, etc.).

6.2. Selección de parámetros y estudio de ablación

En esta sección presentaremos las gráficas mencionadas en el apartado anterior para cada uno de los dos análisis y comentaremos los resultados obtenidos. Y haremos un estudio de cómo afectan los diferentes *scores*, y sus pesos relativos a cada métrica para ambos casos.

6.2.1. Análisis basado en respuestas

Observando los resultados en las gráficas mostradas (figuras 6.1 a 6.8), en primer lugar, se puede ver que la métrica AUC se mantiene bastante estable en torno a valores

iguales o superiores a 0.61, con una ligera mejora al introducir mayor peso al *score* normal que al extendido (por ejemplo, en las combinaciones *score_b05* y *score_b1* con $\alpha = 0{,}9$, $\beta = 0{,}1$). Sin embargo, cuando el nivel de *score* extendido aumenta y su peso también (α cercano a 0 y β a 1), esta métrica no mejora significativamente y tiende a estancarse. Esto indica que el modelo se beneficia de la contextualización que aporta el *score* extendido, pero de una manera bastante limitada

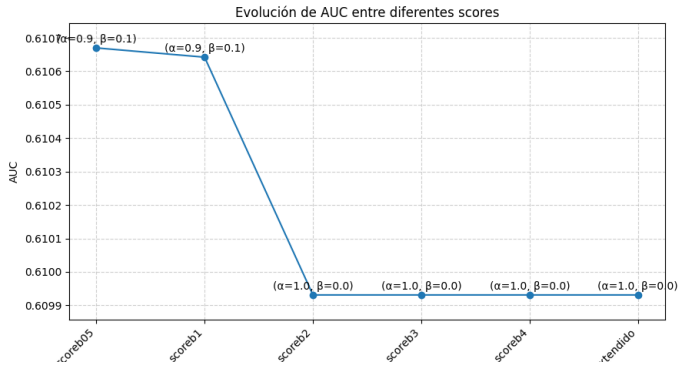


Figura 6.1: Gráfica que muestra la evolución para el *AUC*

En cuanto al *accuracy*, tenemos el caso contrario. Las mejoras son más visibles cuando se da mayor peso al *score* extendido. Las combinaciones que alcanzan el mejor valor (hasta 0.575) son *score_b3*, *score_b4* y *score_extendido*; además, todas ellas presentan valores bajos de α y altos de β, lo que evidencia una clara aportación positiva del *score* extendido. Incluso, para el caso de *score_b4* podemos ver que la mejor opción es anular totalmente α y quedarnos con $\beta = 1$

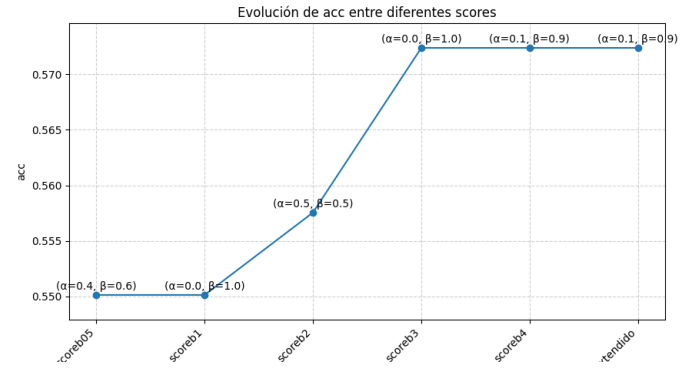


Figura 6.2: Gráfica que muestra la evolución para el *Accuracy*

Esto también se refleja en la precisión negativa y positiva: las variantes con mayor nivel de extensión y con mayor peso (β) para las mismas son las que mejor precisión alcanzan en ambas clases, llegando a 0.575 en *pred_neg* y a 0.55 en *pred_pos*. La simetría en la mejoría de estos dos casos sugiere que el uso del *score* extendido permite al modelo mantener un buen equilibrio en las predicciones, al menos en términos de la métrica de precisión.

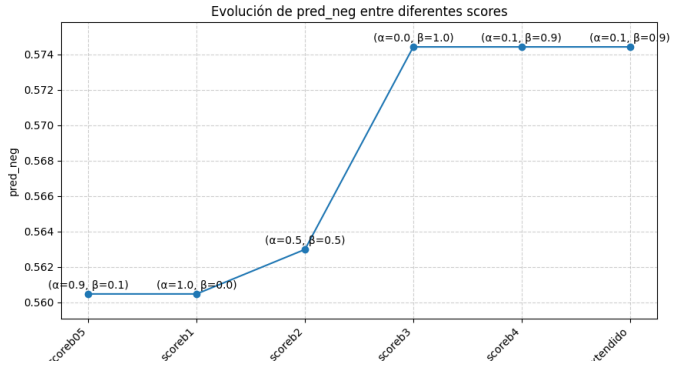


Figura 6.3: Gráfica que muestra la evolución para la precisión negativa

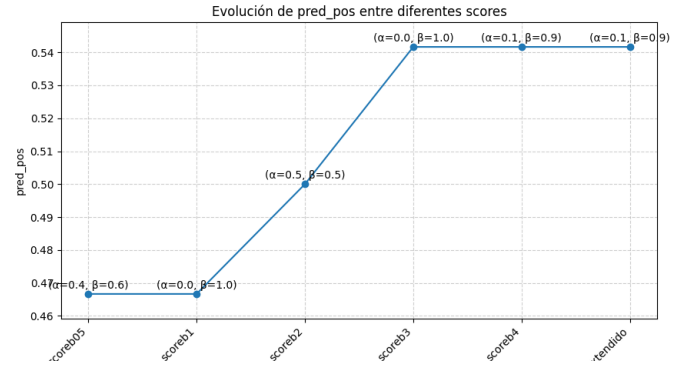


Figura 6.4: Gráfica que muestra la evolución para la precisión positiva

Respecto al *recall*, el comportamiento es mucho más desbalanceado. El *recall* negativo es muy alto en todos los casos, lo cual indica que el modelo identifica con gran fiabilidad los tweets de dicha polaridad. En cambio, para cualquier nivel de *score*, el *recall* positivo es notablemente bajo. Aunque hay que decir que ambos valores del *recall* siguen un patrón similar en tanto a que sí se aprecia una ligera mejora en las combinaciones con mayor *score* extendido, como *score_b3*, *score_b4* y *score_extendido*, que alcanzan un valor de 0.135 frente al 0.113 obtenido en combinaciones más simples para el caso negativo, y un salto de 0.910 a 0.923 para los positivos. Aunque esta diferencia pueda parecer pequeña, resulta significativa considerando el fuerte desbalance entre clases, y quiere decir que las combinaciones más enriquecidas por el *score* extendido ayuda un poco a balancear la predicción. Además de eso es importante mencionar, que los mejores resultados obtenidos con el *score* extendido son aquellos con valores β igual 1, por lo que la información obtenida por el *score* normal no ayuda al balanceo de clases.

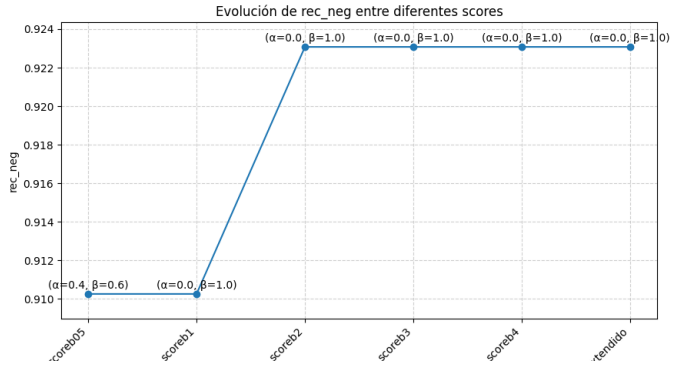


Figura 6.5: Gráfica que muestra la evolución para el *recall* negativo

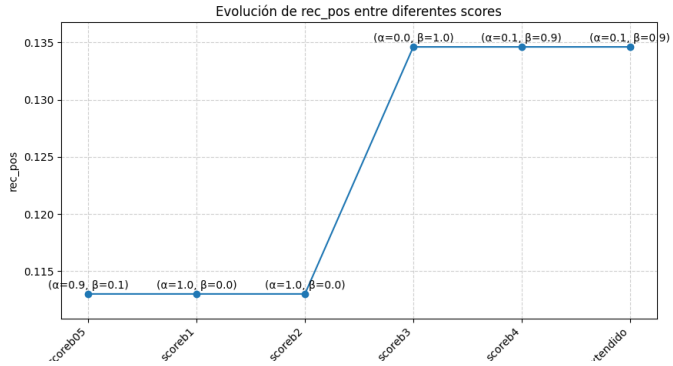


Figura 6.6: Gráfica que muestra la evolución para el *recall* positivo

En cuanto a los *F1-Scores*, se observa una tendencia prácticamente igual que la vista en el *recall*. Para la clase negativa, los valores más altos (hasta 0.707) se obtienen con *scores* más extendidos, y lo mismo sucede con el *F1-Score* positivo, que pasa de 0.181 a 0.204 al utilizar combinaciones más enriquecidas contextualmente. Esto refuerza la recurrente idea de que el *score* extendido ayuda a capturar mejor los casos positivos, incluso si hay un desbalanceo entre polaridades en dicha métrica. Por último, y al igual que sucedía con el *recall*, los mejores resultados obtenidos son aquellos con valores de β cercanos a 1.

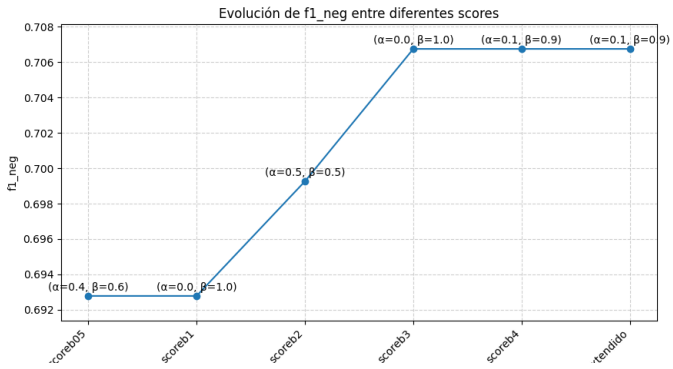


Figura 6.7: Gráfica que muestra la evolución para el *F1-Score* negativo

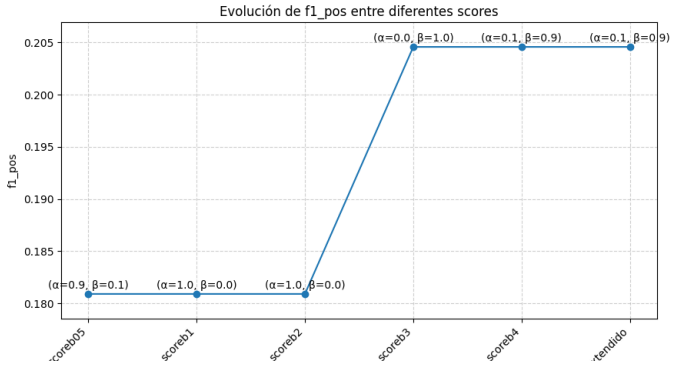


Figura 6.8: Gráfica que muestra la evolución para el *F1-Score* positivo

En conjunto, los resultados sugieren que el uso del contexto añadido gracias al *score* extendido ayuda a mejorar la predicción. Una conclusión importante es que, para *scores* extendidos bajos como b05 o b1, el componente α (*score* normal) tiene mucho más peso que β (*score* extendido), lo cual sugiere que una extensión mínima no aporta ventajas grandes. Sin embargo, a medida que aumenta el nivel de extensión, también aumenta el impacto del componente contextual, aunque esto se estabiliza en torno a *score_b4*, donde se alcanza el pico y el modelo se estanca y no consigue mejorar más.

Como observación adicional, cabe destacar que cada conjunto de 11 respuestas parte de un mismo tweet, por lo que es razonable suponer que estas respuestas puedan compartir una polaridad similar. Esto hace que exista desbalanceo de polaridades dentro de un mismo bloque de respuestas. Además, dado que estamos tratando con temas polémicos de redes sociales, en estos casos, es común que las respuestas reflejen opiniones más bien negativas que positivas, lo que se alinea con el fuerte desbalance observado en el *recall* y el *F1-Score*. Sin embargo, el hecho de que la precisión se mantenga equilibrada entre clases indica que el modelo logra mantener una capacidad de predicción sólida a pesar de este desequilibrio de base.

6.2.2. Análisis de sentimiento puro

Al observar los resultados obtenidos en el análisis del sentimiento puro (mostrado en las figuras 6.9 a 6.16), se aprecia que la métrica *AUC* se mantiene en valores relativamente altos y estables en todas las variantes, oscilando entre 0.747 y 0.752. En este caso, las mejores puntuaciones se alcanzan en *score_b3* y *score_extendido*, con un valor máximo de 0.752, siendo estas combinaciones aquellas donde el *score* extendido tiene mayor peso. No obstante, la diferencia es muy pequeña respecto a variantes con alto valor de α, lo que indica que, a diferencia del caso anterior, el impacto del *score* extendido en la *AUC* es limitado y el *score* normal (basado únicamente en el área de cada palabra) mantiene un papel importante, siendo dominante en los casos de b05, b1 y b2 como antes.

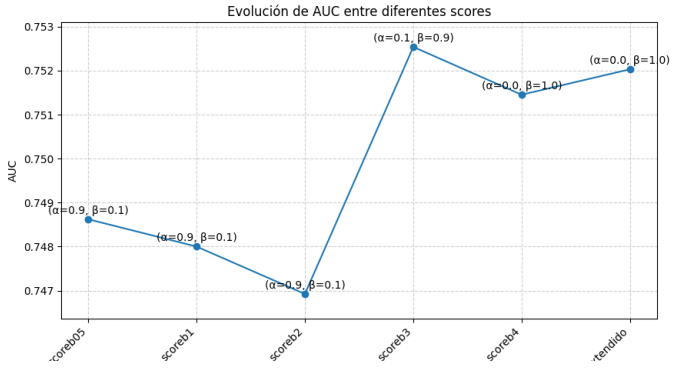


Figura 6.9: Gráfica que muestra la evolución para el AUC

En cuanto al *accuracy*, este se mantiene constante en 0.72 para todas las combinaciones, ya que la mejor combinación es siempre aquella en la que $\alpha = 1$ y $\beta = 0$. Esto sugiere que el *score* normal ya es lo suficiente potente para alcanzar la máxima precisión posible permitida por el modelo, y que el *score* extendido no ayuda a mejorarla. Esta estabilidad también puede interpretarse como una señal de que este conjunto de datos está más equilibrado que el anterior, cosa que se corroborará en las siguientes métricas

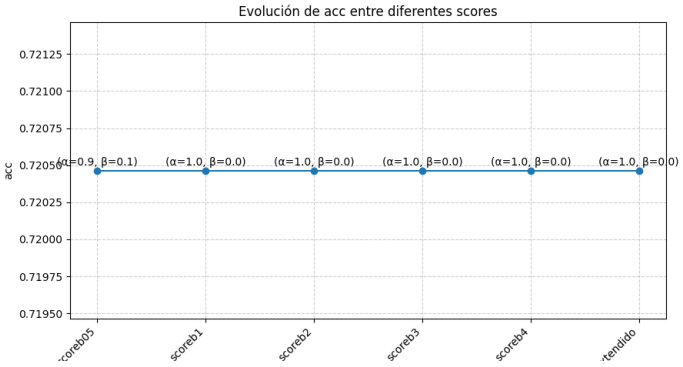


Figura 6.10: Gráfica que muestra la evolución para el *Accuracy*

Analizando la precisión, se observan ligeras mejoras en *pred_neg* al introducir cierto grado de *score* extendido, alcanzando valores de hasta 0.840 en *score_b4*, aunque también se obtienen buenos resultados con combinaciones de poca extensión y de α altas. Por otro lado, la precisión positiva (*pred_pos*) permanece completamente constante en 0.674 para todas las variantes con $\alpha = 1$, igual que nos pasaba en la *accuracy*.

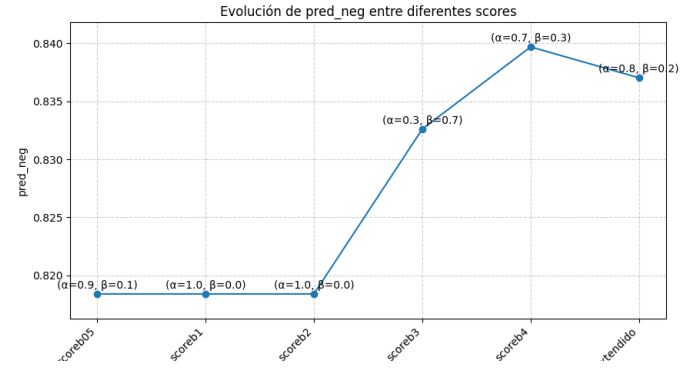


Figura 6.11: Gráfica que muestra la evolución para la precisión negativa

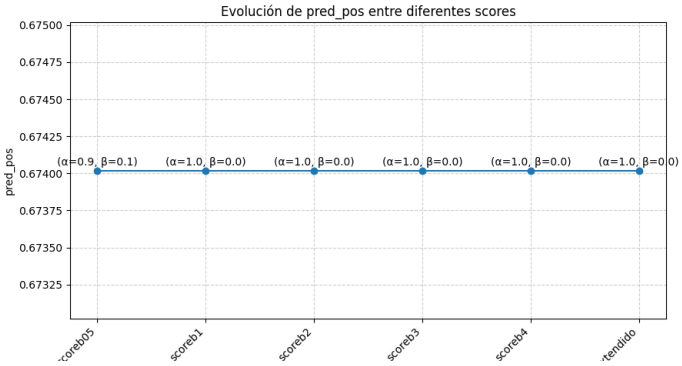


Figura 6.12: Gráfica que muestra la evolución para la precisión positiva

Respecto al *recall*, los valores presentan cierto desblanceo por primera vez en este conjunto de datos, pero de forma mucho más controlada que en el anterior. Aquí, el *recall* positivo es significativamente superior al negativo, alcanzando hasta 0.91 con *scores* extendidos (*score_b3*, *score_b4* y *score_extendido*). Sin embargo, el *recall* negativo aguanta y permanece estancado en 0.561, sin variación alguna en ninguna de las combinaciones, pero no siempre con valores $\alpha = 1$ y $\beta = 1$ como antes, sino que vemos combinaciones de $\alpha = 0{,}5, \beta = 0{,}5$ y $\alpha = 0{,}9, \beta = 0{,}1$ para los *scores* b05 y b1 respectivamente, lo cual es buena noticia.

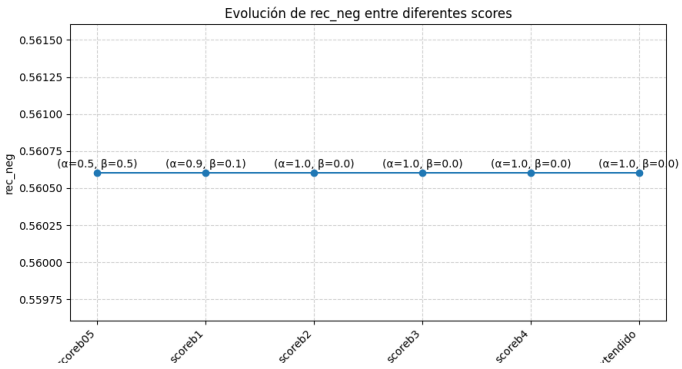


Figura 6.13: Gráfica que muestra la evolución para el *recall* negativo

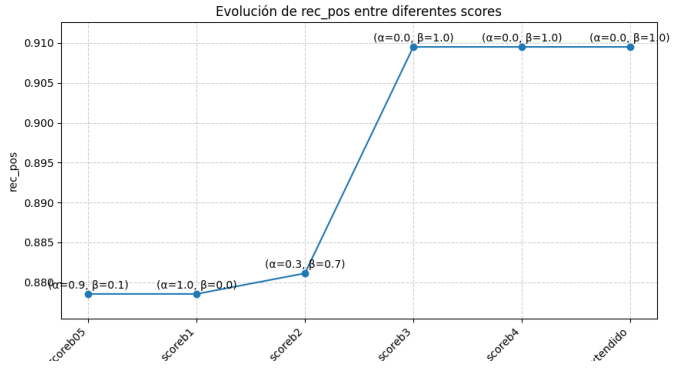


Figura 6.14: Gráfica que muestra la evolución para el *recall* positivo

En cuanto a los *F1-Score*, se confirma el patrón anterior: el F1 negativo permanece fijo en 0.662 para todas las variantes, mientras que el F1 positivo alcanza 0.761 en todos los casos (en ambas situaciones *alpha* = 1 menos para el *score* b05 que es 0.9) Esta estabilidad, aunque pueda parecer poco informativa, reafirma que en este conjunto de datos no existe un gran margen de mejora a través de la incorporación de contexto adicional. El modelo ya capta adecuadamente la información semántica de cada tweet sin requerir extensión vertical del mapa de calor. Además de eso, estas dos métricas, el *recall* y el *F1-Score*, nos indican que aunque pueda existir desbalanceo de polaridades, es de manera mucho más controlada.

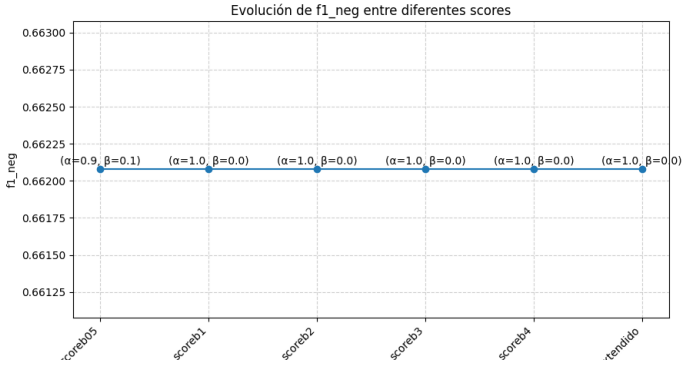


Figura 6.15: Gráfica que muestra la evolución para el *F1-Score* negativo

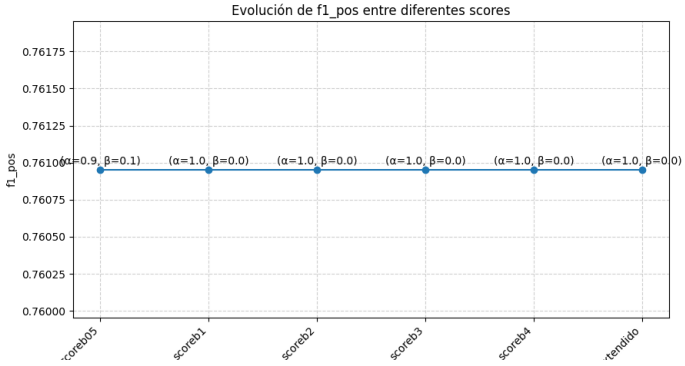


Figura 6.16: Gráfica que muestra la evolución para el *F1-Score* positivo

6.2.3. Discusión

En conclusión, podría decirse los resultados muestran que el *score* normal (basado únicamente en la información local de cada palabra) es suficiente para alcanzar un rendimiento alto y estable en la clasificación de sentimiento puro. A diferencia del análisis basado en respuestas, aquí la inclusión de contexto mediante *scores* extendidos aporta menos mejoras, y cuando lo hace, son mínimas. Esto puede deberse a que los tweets analizados son independientes entre sí, sin un vínculo explícito con el contenido que están respondiendo como antes. Por lo tanto, el significado de las palabras en cada tweet se mantiene relativamente autónomo y estable, y el modelo puede capturar la polaridad sin requerir mayor información adicional del entorno.

Otro factor clave y que diferencia claramente este conjunto de datos del anterior es el aparente equilibrio entre clases. Mientras que en el análisis basado en respuestas existía un fuerte desbalance hacia la clase negativa, aquí todas las métricas (includo las que están un poco más desbalanceadas) muestran valores similares entre clases. Esto sugiere que el conjunto de datos de sentimiento puro es más balanceado, y por ello el modelo no se ve obligado a especializarse en una clase predominante. En consecuencia, la extensión del contexto pierde relevancia frente a la información directa del *score* normal, lo cual explica por qué α sigue teniendo un peso dominante en casi todas las combinaciones exitosas.

6.3. Comparación con otros trabajos

Viendo los resultados de nuestro modelo, podría decirse que el caso de análisis de sentimiento puro es el mejor, y que dentro de él, los casos en el que el *score* normal tiene mayor peso son los que más destacan. Pues partiendo de eso, vamos a comprar dichos resultados con los obtenidos con los modelos BERT. *"BERT es una técnica basada en redes neuronales para el pre-entrenamiento del procesamiento del lenguaje natural desarrollada por Google"* (Tensorflow, 2022).
Para realizar las pruebas con estos modelos de lenguaje, no hemos necesitado desarrollar código alguno, pues la implementación se ha podido obtener de esta página Clasificar texto con BERT (Tensorflow, 2022). En la que se puede cambiar fácilmente el conjunto de datos a estudio, así como el modelo *BERT* a utilizar. Esto último es posible debido a un extenso diccionario que permite elegir un modelo entre una larga lista (*BERT*, *ELECTRA*, *ALBERT*, etc.).

Para nuestro estudio hemos elegido los 4 siguientes modelos descritos en la tabla 6.2 como referencia

Tabla 6.2: Modelos BERT elegidos

Modelo	capas	neuronas por cada	cabezas de atención
BERT L2 H128 A2	2	128	2
BERT L4 H128 A2	4	128	2
BERT L2 H256 A4	2	256	4
ELECTRA	-	-	-

Dicho esto, presentaremos los resultados en gráficas (figuras 6.17 a 6.24 para el caso

de análisis basado en respuesta, y figuras 6.25 a 6.32 para el caso de análisis de sentimiento puro) como antes. Pero recordamos lo dicho en el capítulo de experimentación. En estos modelos preentrenados no hemos podido incluir información derivada del rastreo ocular, por lo que no hay *scores*, ni α's ni β's. Es decir, los valores son los que son. Antes de comenzar con la presentación de resultado, es importante mencionar que hemos incluido una barra horizontal roja en cada gráfica que representa el mejor valor obtenido para dicho análisis y métrica con nuestro modelo, para que esto nos sirva como una comparativa directa y visual.

6.3.1. Análisis basado en respuestas

Empezando por el *AUC* este caso se puede ver que todos los valores oscilan en el 0.5, con el electra siendo un poco superior con 0.52. Ese mismo patrón se repite en el acc, pero esta vez el modelo *L2-H128-A2* es el mejor, y en algunos casos como *L4-H128-A2* vemos un valor de 0.4. Estas dos métricas ya nos dicen que los modelos preentrenados de BERT no sirven para nuestros conjuntos de datos, y que la información del rastreo ocular era necesaria. Y si vemos el resto de métricas, esto se confirma.

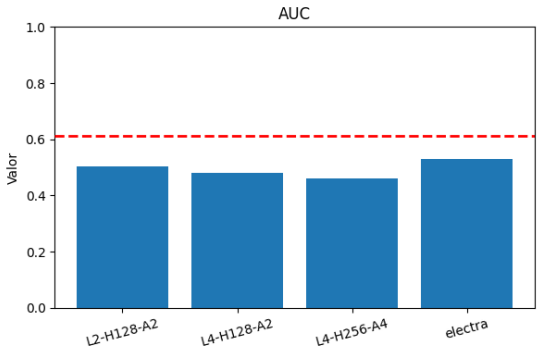


Figura 6.17: Gráfica que muestra la evolución para el *AUC*

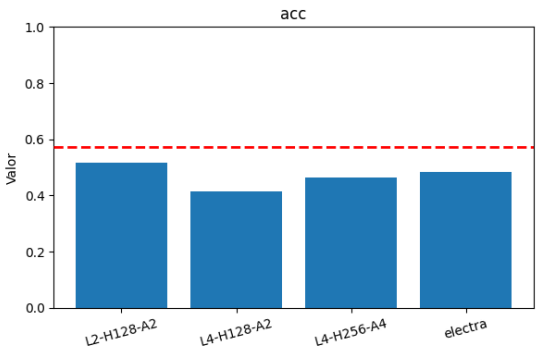


Figura 6.18: Gráfica que muestra la evolución para la *Accuracy*

Por ejemplo, para la precisión negativa, de nuevo los valores oscilan el 0.5, excepto el modelo *L2-H128-A2* que llega hasta 0.6. Sin embargo, la positiva se hunde a valores entre 0.25 y 0.40, excepto para electra, que llega a 0.6. Sin embargo, esto no nos vale, hay un desbalanceo de clases, pero también de modelos, cosa que antes no pasaba con los *scores* extendidos.

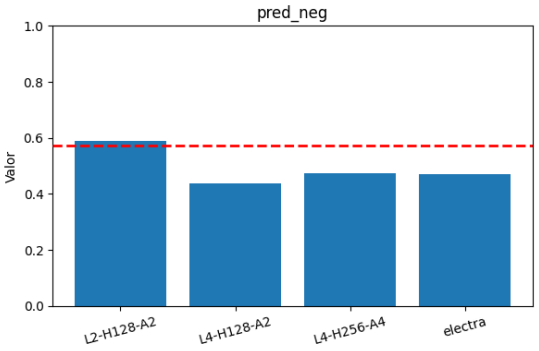


Figura 6.19: Gráfica que muestra la evolución para la precisión negativa

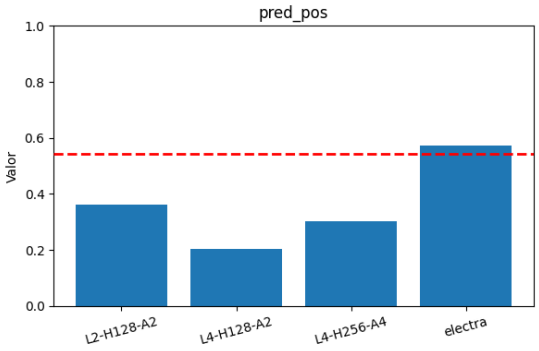


Figura 6.20: Gráfica que muestra la evolución para la precisión positiva

El *recall* negativo, por primera vez en este conjunto de datos, da datos relativamente buenos, con valores entre 0.6 y 0.7, siendo el mejor *L4-H256-A4*, pero claro, esto a costa de hundir el *recall* positivo a valores de 0.2 y de 0.4 solo para *L2-H128-A2*.

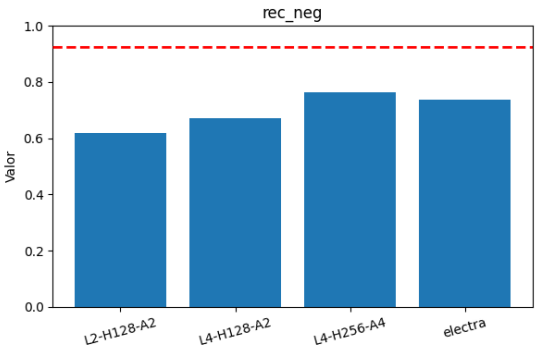


Figura 6.21: Gráfica que muestra la evolución para el *recall* negativo

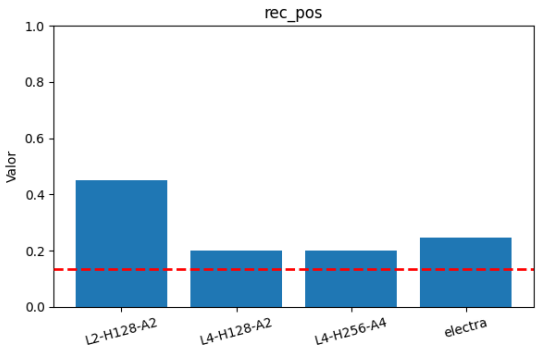


Figura 6.22: Gráfica que muestra la evolución para el *recall* positivo

Para el FI-score parece algo más balanceado, con valores en torno al 0.5 para todos los modelos, tanto para la clase negativa como para la positiva, siendo quizás electra el que mejor balancea, pero sin que sea algo reseñable.

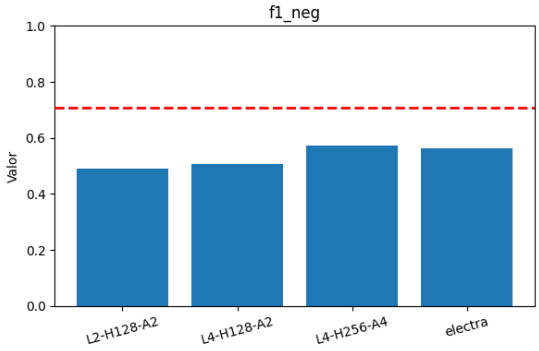


Figura 6.23: Gráfica que muestra la evolución para el *F1-Score* negativo

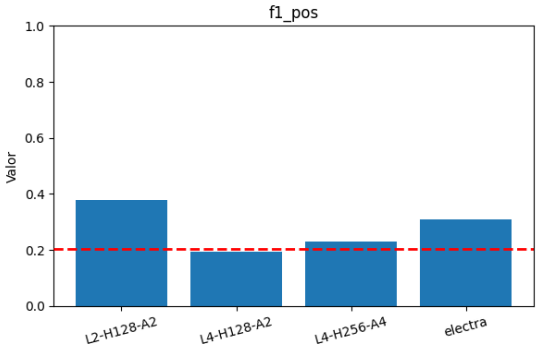


Figura 6.24: Gráfica que muestra la evolución para el *F1-Score* positivo

En conclusión, queda claro que estos modelos preentrenados de BERT no sirven para nuestro conjunto de datos textual sin información visual añadida, al menos para el análisis basado en respuestas, veamos que pasa con el análisis de sentimiento puro.

6.3.2. Análisis de sentimiento puro

No entraremos mucho en detalle para los resultados, porque parecen bastante similares a los del análisis anterior. Primero, para *AUC* todos los valores son de 0.5 menos el primer modelo *L2-H128-A2* que es de 0.42. Para la *accuracy*, parece que todos los modelos tienen un valor neutro de 0.5

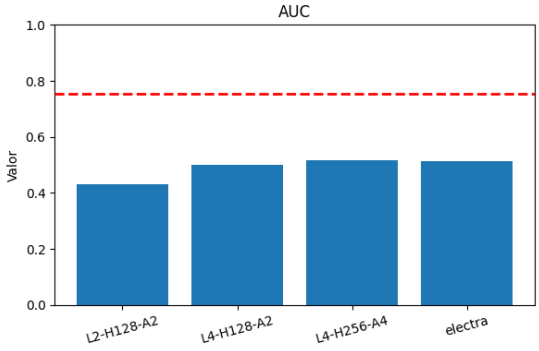


Figura 6.25: Gráfica que muestra la evolución para el *AUC*

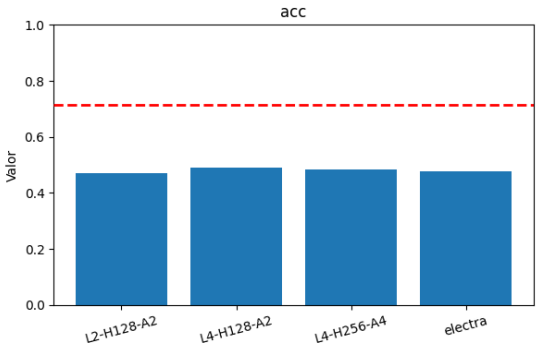


Figura 6.26: Gráfica que muestra la evolución para la *Accuracy*

Para la precisión negativa, seguimos con valores en torno al 0.5, pero ligeramente inferior, sobre unos 0.47-0.48. Sin embargo, la precisión positiva si es buena, con valores de 0.6-0.65, menos para el modelo *L2-H128-A2* que apenas tiene 0.3.

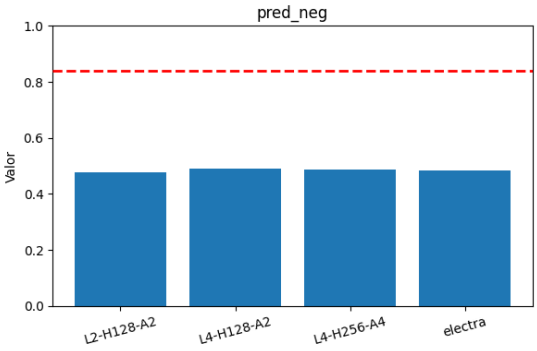


Figura 6.27: Gráfica que muestra la evolución para la precisión negativa

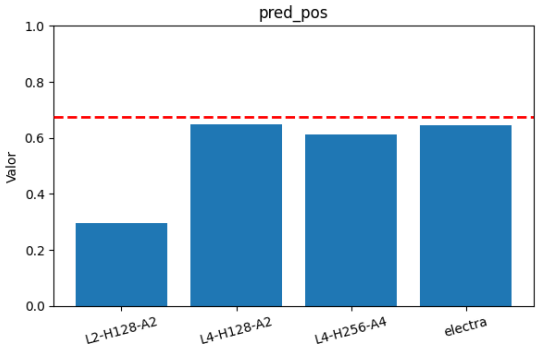


Figura 6.28: Gráfica que muestra la evolución para la precisión positiva

En el *recall*, igual que en el conjunto de datos anterior, hay mucho desbalanceo, con valores de 0.15-0.2 en el caso positivo para todos los modelos, y de 0.8-0.85 para los negativos. Aquí es importante recordar que en el análisis de sentimiento para nuestros conjuntos de datos, al menos conseguiamos paliar el desbalanceo de clases, pero aquí no.

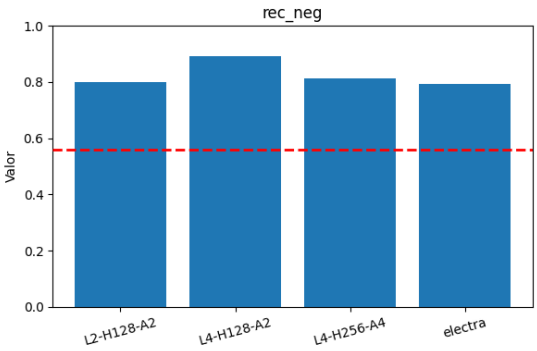


Figura 6.29: Gráfica que muestra la evolución para el *recall* negativo

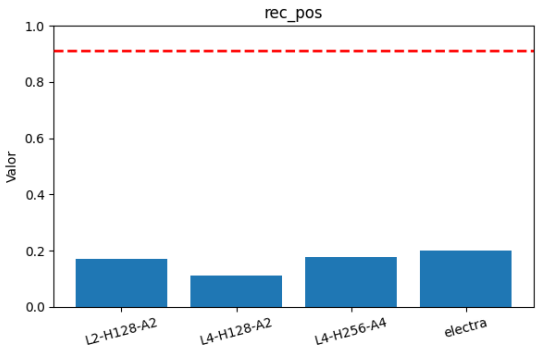


Figura 6.30: Gráfica que muestra la evolución para el *recall* positivo

Con el *F1-Score* pasa algo similar pero no tan abruptamente, con valores de 0.2 para la clase negativa y de 0.6 para la positiva

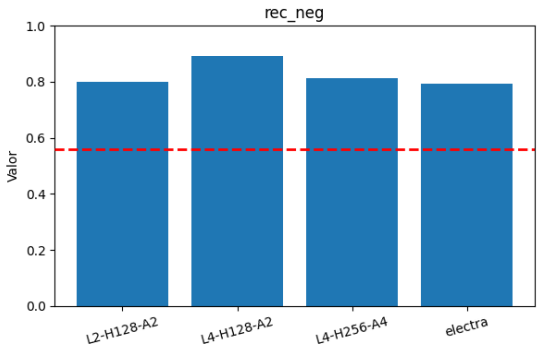


Figura 6.31: Gráfica que muestra la evolución para el *F1-Score* negativo

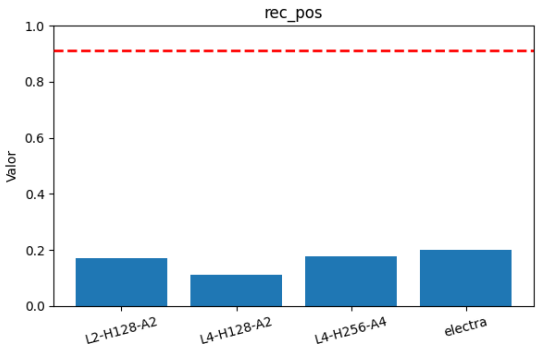


Figura 6.32: Gráfica que muestra la evolución para el *F1-Score* positivo

En conclusión, el conjunto de datos basado únicamente en la información textual de los tweets, combinado con modelos complejos preentrenados no funciona. De aquí se derivan dos cosas: Primero, la información contextual aportada por los mapas de calor era muy necesaria. Segundo: para unos conjuntos de datos tan pequeños y simples, funciona mucho mejor una red neuronal simple que modelos complejos y preentrenados, que brillarían más en conjuntos de datos grandes de miles y miles de datos.

Capítulo 7

Conclusiones y trabajo futuro

Para este último apartado, vamos a ver las conclusiones finales, así como el trabajo futuro que podría ampliar este TFM.

7.1. Conclusiones

Recordemos brevemente los tres objetivos iniciales marcados en el capítulo 1:

1. Construcción de un conjunto de datos enriquecido con información visual.
2. Desarrollo de un modelo capaz de analizar sentimiento en un conjunto de datos como el presentado.
3. Evaluación y comparación del modelo propuesto.

1. Construcción de un conjunto de datos enriquecido con información visual.
Una de las conclusiones más claras de este trabajo es que contar con un buen conjunto de datos es fundamental. No se trata tanto de tener muchos datos, sino de que estén bien seleccionados, bien etiquetados y alineados con el objetivo del modelo. En nuestro caso, lo que realmente marcó la diferencia en el rendimiento no fue el modelo en sí, sino las entradas que este recibía.
Herramientas como los mapas de calor y los *scores* visuales resultaron especialmente útiles, ya que aportaban un contexto adicional que un conjunto de datos puramente textual no ofrece. Gracias a esta información visual fue posible enriquecer las muestras y tomar decisiones más fundamentadas. Por tanto, se puede decir que este primer objetivo no solo se ha cumplido, sino que ha sido clave para todo el trabajo.

2. Desarrollo de un modelo capaz de analizar sentimiento en un conjunto de datos como el presentado.
Durante el desarrollo del modelo se puso el foco en la simplicidad, la interpretabilidad y el aprovechamiento de la información visual. En este sentido, el modelo empleado consiguió integrar la información visual proveniente del rastreo ocular con las representaciones textuales. Todo esto fue gracias a una red neuronal sencilla, con unas métricas de centralidad claras y útiles, unas métricas, que sin los grafos de polaridad, hubiera sido imposible obtener. Por lo que al final se puede decir que se ha conseguido una muy buena integración entre una red neuronal sencilla, grafos de

polaridad, y esa combinación de información textual y visual

3. Evaluación y comparación del modelo propuesto.
En la fase de evaluación, se observó que el análisis de sentimiento puro ofrecía mejores resultados que el análisis centrado en las respuestas. Esto se debe a que las respuestas breves suelen contener polaridades comunes entre bloques, lo que complica la clasificación: El modelo fue capaz de detectar bien la clase negativa, pero tuvo dificultades con las positivas. En el análisis de sentimiento puro, al trabajar con textos independientes, las polaridades están más mezcladas, lo que supone un conjunto de datos más balanceado
Por otra parte, también se probaron enfoques alternativos con modelos más complejos, como BERT. Y aunque en teoría estos modelos deberían haber ofrecido buenos resultados, en la práctica no funcionaron bien. La complejidad de BERT no se tradujo en mejor rendimiento, en gran parte porque no era capaz de aprovechar la información visual que sí se incluía en los otros enfoques. Esto refuerza la idea de que más complejo no siempre es mejor, especialmente cuando el modelo no está alineado con el tipo de datos y estos no tienen la suficiente riqueza.

Lo más importante que se ha aprendido es que, en este tipo de tareas, el diseño del conjunto de datos puede ser más determinante que la complejidad del modelo. Enriquecer los datos con señales relevantes como los mapas de calor generados por el rastreo ocular aporta un valor añadido que los modelos más sofisticados no siempre son capaces de aprovechar por sí solos. A veces, simplificar y entender bien el problema da mejores resultados que aplicar técnicas avanzadas sin un buen soporte en los datos.

7.2. Líneas de trabajo futuro

Una línea de trabajo futuro bastante interesante sería combinar el *eye tracking* con otras formas de análisis emocional para enriquecer la información que se tiene del usuario. Es decir, no quedarnos solo con saber dónde está mirando, sino también como se siente mientras lo hace. Para eso, se pueden usar técnicas como análisis facial (expresiones, microgestos...), detección del tono de voz, o incluso sensores biométricos (frecuencia cardíaca, sudoración, etc.).

Al combinar estas fuentes de datos, se podría tener un sistema mucho más completo que no solo entienda lo que llama la atención del usuario, sino también si esa atención es positiva, negativa, de interés, de aburrimiento, etc. Por ejemplo, si un usuario está mirando un punto concreto de una interfaz, pero al mismo tiempo tiene una expresión de confusión o frustración, el sistema podría reaccionar en consecuencia.

Esto permitiría adaptar la interfaz en tiempo real, cambiando tanto el contenido como la forma en la que se presenta. En contextos como la educación, por ejemplo, sería útil para saber si un alumno está realmente entendiendo el contenido o si está simplemente mirando por mirar. Si se detecta que está confundido o aburrido, se podría cambiar el tipo de explicación, añadir ejemplos, o incluso introducir un pequeño descanso. Lo mismo se puede aplicar en otros ámbitos como la accesibilidad, para hacer las interfaces más inclusivas y adaptadas a personas con diferentes ca-

pacidades, o en marketing digital, para entender mejor la reacción emocional que provoca un anuncio o un producto.

También sería interesante explorar cómo afecta el contexto a todas estas variables. Por ejemplo, no es lo mismo analizar a un usuario que está solo en casa, que a uno que está en una sala llena de gente. El entorno influye mucho en cómo nos sentimos y cómo interactuamos con la tecnología, así que sería interesante ver cómo se puede tener eso en cuenta también en los modelos.

Un par de aplicaciones de estas técnicas podrían ser:

1. **Optimización de campañas de marketing y diseño de interfaces de usuario**: Las empresas pueden evaluar cómo los usuarios interactúan visualmente con elementos de interés y ajustar su estrategia en función de los resultados obtenidos.

2. **Análisis más profundo de la interacción en redes sociales**: Esta técnica permite no solo conocer las opiniones de los usuarios, sino también identificar el porqué, viendo qué contenidos captan más su atención y cómo influyen en función de su relevancia.

En resumen, el siguiente paso lógico sería avanzar hacia un sistema más multimodal, que no se base solo en lo que el usuario hace, sino también en cómo lo hace y cómo se siente mientras lo hace. Esto permitiría crear interfaces mucho más inteligentes, humanas y adaptativas.

Anexo

En este anexo adjuntamos el código desarrollado para la realización del trabajo.
Enlace : https://github.com/AitorJuncal/TFM

Referencias

admin@graphonline.ru (2015). Graph online creator - graphonline.ru. [Last accessed: 2025-09-01].

Contributors, W. (2024). Modelo de color hsv — wikipedia, la enciclopedia libre. [Internet; descargado 10-diciembre-2024].

Jayoma, J. M., Moyon, E. S., and Morales, E. M. O. (2020). Ocr based document archiving and indexing using pytesseract: A record management system for dswd caraga, philippines. In *2020 IEEE 12th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)*, pages 1–6. IEEE.

Lim, J. Z., Mountstephens, J., and Teo, J. (2020). Emotion recognition using eye-tracking: Taxonomy, review and current challenges. *Sensors*, 20(8).

Lu, L. and Zhang, M. (2013). *Edge Betweenness Centrality*, pages 647–648. Springer New York, New York, NY.

Lu, Y., Zheng, W.-L., Li, B., and Lu, B.-L. (2015). Combining eye movements and eeg to enhance emotion recognition. In *IJCAI*, volume 15, pages 1170–1176. Buenos Aires.

Muñoz, G. F., Casero, J. G., and Gilaberte, R. L. (2024). Visualkine: Revolucionando el análisis de la atención visual y emocional para el neuromarketing. In AIPO, editor, *Interacción '24: Proceedings of the XXIV International Conference on Human Computer Interaction*, page 67, New York, NY, USA. Association for Computing Machinery.

Niehorster, D. C., Hessels, R. S., and Benjamins, J. S. (2020). Glassesviewer: Open-source software for viewing and analyzing data from the tobii pro glasses 2 eye tracker. *Behavior Research Methods*, 52(3):1244–1253.

Nigro, H. (2020). Una revisión a la minería de opiniones y los retos del pnl. *Revista Ingeniería, Matemáticas y Ciencias de la Información*, 7(13):105–110.

Oliva, M. and Anikin, A. (2018). Pupil dilation reflects the time course of emotion recognition in human vocalizations. *Scientific reports*, 8(1):4871.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Pro, T. (2019). Tobii pro nano – product description. https://www.srlabs.it/wp-content/uploads/2019/10/Tobii_Pro_Nano.pdf. [Last Accessed: 2025-09-01].

Raudonis, V., Dervinis, G., Vilkauskas, A., Paulauskaitė-Tarasevičienė, A., and Keršulytė-Raudonė, G. (2013). Evaluation of human emotion from eye motions. *International journal of advanced computer science and applications.*, 4(8):79–84.

Tensorflow (2022). Clasificar texto con bert. https://www.tensorflow.org/text/tutorials/classify_text_with_bert?hl=es-419. [Last Accessed: 2025-09-01].

Tsang, V. (2018). Eye-tracking study on facial emotion recognition tasks in individuals with high-functioning autism spectrum disorders. *Autism*, 22(2):161–170.

Turc, I., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Well-read students learn better: On the importance of pre-training compact models. *arXiv preprint arXiv:1908.08962v2*.

Van Mieghem, P. (2014). Graph eigenvectors, fundamental weights and centrality metrics for nodes in networks. *arXiv preprint arXiv:1401.4580*.

Wan Adnan, W. A., Hassan, W. N. H., Abdullah, N., and Taslim, J. (2013). Eye tracking analysis of user behavior in online social networks. In *International Conference on Online Communities and Social Computing*, pages 113–119. Springer.

Wang, Y., Lv, Z., and Zheng, Y. (2018). Automatic emotion perception using eye movement information for e-healthcare systems. *Sensors*, 18(9):2826.

Xu, G., Meng, Y., Qiu, X., Yu, Z., and Wu, X. (2019). Sentiment analysis of comment texts based on bilstm. *Ieee Access*, 7:51522–51532.

Zheng, W.-L., Dong, B.-N., and Lu, B.-L. (2014). Multimodal emotion recognition using eeg and eye tracking data. In *2014 36th annual international conference of the IEEE engineering in medicine and biology society*, pages 5040–5043. IEEE.